Making Model Runs Auditable by Default: A Design Science Study of Signed, Replayable Capability Receipts for Organizational AI Accountability

Lakshman Turlapati[1]
Rabih Neouchi[2]

[1]Full Self Browsing

[2]Naveen Jindal School of Management, The University of Texas at Dallas, Richardson, TX, USA

Corresponding author: Lakshman Turlapati, alumnus, Naveen Jindal School of Management, The University of Texas at Dallas (M.S., December 2025).

Email: lakshmanturlapati@gmail.com

Abstract

Organizations embed large language models in business processes, yet auditors need evidence of which model served a run and what it produced. This design-science study states five requirements and four principles for run-level accountability and instantiates them in Lattice, an open-source runtime. When a signer is configured, single calls and multi-agent crews yield canonical receipts, signed after matched personal data is masked. A receipt commits to hashes of the input artifacts and the output, to the output contract, and to the selected route, but not to the task text, the policy, or the reasons for the route. An offline evaluation on mocked providers rejects tampering of single receipts, binds recorded outputs to the signed output hash, and adds milliseconds of signing cost. Freshness binding and open-chain length checks stay verifier opt-ins, and crew chains list their members. The study contributes design knowledge for accountable AI information systems.

Keywords: AI accountability; Design science research; Audit trail; Large language models; Provenance; AI governance

1 Introduction

Generative artificial intelligence (AI) has moved from experimentation into operational information systems that approve refunds, triage claims, summarize case files, and draft regulated communications (Berente et al., 2021; Bommasani et al., 2021). These systems combine text, images, documents, and tool calls, and they often route a single task across several model providers for reasons of cost, latency, privacy, or capability (Chen et al., 2023; Ong et al., 2024). Their outputs therefore carry organizational consequences, and with consequences comes accountability: someone must be able to explain, after the fact, what the system was

asked to do, which model served the request under which constraints, and what came back (Wieringa, 2020; Novelli et al., 2023).

Regulatory and standards frameworks make this expectation explicit. The European Union Artificial Intelligence Act requires providers of high-risk systems to enable automatic recording of events over the system's lifetime and to keep such logs (European Parliament and Council of the European Union, 2024). The NIST AI Risk Management Framework asks organizations to document and monitor AI systems so that risks can be measured and managed (Tabassi, 2023), and ISO/IEC 42001 specifies a management system in which AI-related processes must be controlled and evidenced (ISO/IEC, 2023). Scholarship on AI auditing reaches a similar conclusion: internal and external audits depend on artifacts that make claims about a system verifiable rather than merely asserted (Raji et al., 2020; Brundage et al., 2020; Mökander and Floridi, 2021; Mökander et al., 2023).

Current practice falls short of this. Most organizations integrate models through provider wrappers and orchestration libraries that normalize request formats and add conveniences such as tool calling and tracing (Amershi et al., 2019; Sculley et al., 2015). Their traces are written for human debugging. A trace can be edited after the fact, it does not bind inputs to outputs, and a third party cannot check it without trusting the party that produced it (Schneier and Kelsey, 1999; Crosby and Wallach, 2009). The result is an accountability gap between what governance frameworks expect and what AI-enabled information systems actually record (Raji et al., 2020; Papagiannidis et al., 2025).

This study addresses that gap through design science research (DSR) (Hevner et al., 2004; Peffers et al., 2007; Gregor and Hevner, 2013). The research question is: *How should a model-run infrastructure be designed so that every run of an AI-enabled information system yields evidence that auditors can verify independently and operators can reproduce, at acceptable cost?* We derive design requirements from the governance and auditing literature, formulate design principles, instantiate them in an open-source artifact called Lattice (Turlapati, 2026), and evaluate the artifact against each requirement with a reproducible, offline evaluation harness. The central construct is the *capability receipt*: a per-run record that is canonicalized, redacted before signing, signed with a public-key signature, and linked to content-addressed artifacts so that the record and the stored artifacts can be checked offline.

The study makes three contributions. First, it specifies requirements and principles for run-level AI accountability, a nascent design theory in the sense of Gregor and Jones (2007) that is independent of any particular implementation. Second, it provides an instantiation whose behavior is evaluated quantitatively, covering integrity under attack, binding of recorded outputs, policy enforcement, and runtime overhead. Third, it reports where the instantiation does *not* meet its requirements, findings that are themselves design knowledge for builders of accountable AI systems (Baskerville et al., 2018). We position the contribution as an *improvement* in the sense of Gregor and Hevner (2013): a known problem, auditable record keeping, addressed with a new solution for a new class of systems, multi-provider model runs.

The remainder of the paper follows the DSR publication schema proposed by Gregor and Hevner (2013). Section 2 reviews related work. Section 3 describes the research approach. Section 4 derives requirements and principles. Section 5 presents the artifact. Section 6 reports the evaluation. Section 7 discusses contributions, implications, and limitations, and Section 8 concludes.

2 Background and Related Work

2.1 AI accountability, governance, and auditing

Accountability in AI denotes the obligation of an actor to explain and justify the conduct of a system to a forum that can pass judgment and impose consequences (Wieringa, 2020; Novelli et al., 2023). IS research frames AI management as a matter of autonomy, learning, and inscrutability (Berente et al., 2021), and it describes how organizations envelop AI with sociotechnical controls to contain its behavior (Asatiani et al., 2021). Organizational AI governance comprises the structures, processes, and procedures that align AI use with organizational and regulatory goals (Mäntymäki et al., 2022; Schneider et al., 2022; Papagiannidis et al., 2025). Across these accounts, governance depends on evidence. Policies and controls mean little unless an organization can show, for specific decisions, that the controls operated.

The AI auditing literature makes this dependence concrete. Raji et al. (2020) propose an end-to-end internal audit framework and stress documentation artifacts throughout the development life cycle. Mökander and Floridi (2021) argue for ethics-based auditing as a governance mechanism, and Mökander et al. (2023) propose a three-layered approach to auditing large language models at the governance, model, and application layers. Brundage et al. (2020) call for mechanisms, including audit trails, that make claims about AI development verifiable by outsiders. Transparency research cautions that transparency must be designed for specific audiences and purposes (Felzmann et al., 2019; Rai, 2020), and human-centered AI calls for reliable, safe, and trustworthy systems backed by audit trails (Shneiderman, 2020). Documentation instruments such as model cards (Mitchell et al., 2019) and datasheets for datasets (Gebru et al., 2021) standardize disclosure about models and data. However, they describe artifacts at release time, not individual production runs. The application layer of Mökander et al. (2023), where a model is used in a specific business process, is where the evidence gap is widest, and it is the layer this study targets.

2.2 Tamper-evident logging, provenance, and transparency

Computer security offers a long tradition of tamper-evident records. Haber and Stornetta (1991) showed how to time-stamp digital documents, and Merkle (1988) introduced hash-tree commitments. Schneier and Kelsey (1999) designed secure audit logs whose entries cannot be undetectably altered after compromise, and Crosby and Wallach (2009) gave efficient data structures for tamper-evident logs with logarithmic proofs. Certificate Transparency (Laurie et al., 2013) publishes certificates to append-only logs so that misissuance becomes publicly detectable. Software supply chain research applies these ideas to builds. in-toto (Torres-Arias et al., 2019) signs each step of a software supply chain, The Update Framework addresses survivable key compromise (Samuel et al., 2010), Sigstore makes signing and transparency logging accessible to developers (Newman et al., 2022), and SLSA defines levels of supply chain integrity (OpenSSF, 2023). The Dead Simple Signing Envelope (DSSE) specifies a minimal signed envelope with a pre-authentication encoding (Secure Systems Lab, 2021). Data provenance research models the derivation history of data products (Moreau et al., 2011; Herschel et al., 2017).

These lines of work supply mechanisms (canonical encodings, signatures, hash chains, transparency logs) but were not designed for the semantics of a model run. A model run has a declared intent, a routing

decision among providers, policy constraints, output invariants, and multimodal inputs that may contain personal data. Model gateways and observability libraries such as LiteLLM (BerriAI, 2023), LangChain callbacks (LangChain Inc., 2023), OpenLLMetry (Traceloop, 2023), and the Portkey gateway (Portkey AI, 2024) sit on that path and log the calls, but none of the four signs its log, and the logs that could be read stored the prompt text. Section 6.8 reports these logs and the wrappers' latency on a CPU-only host, where generation noise dominates.

2.3 Verifiable inference and model routing

A complementary stream seeks cryptographic guarantees that a specific model computed a specific output, using zero-knowledge proofs (Kang et al., 2022; Sun et al., 2024). Such proofs give strong guarantees but currently require access to model weights and impose high costs, so they do not apply to proprietary hosted models consumed through application programming interfaces (APIs). Work on model routing optimizes cost and quality by selecting among models (Chen et al., 2023; Ong et al., 2024), often with learned routers whose decisions are hard to audit. Security research on LLM-integrated applications shows that model outputs can be manipulated through indirect prompt injection (Greshake et al., 2023), which raises the stakes for run-level evidence.

2.4 Agent receipts and verifiable agent provenance

Three recent proposals are closest to this study. *TrustTrack* is a vision paper that argues for verifiable identity, policy commitments, and tamper-resistant behavioral logs in multi-agent systems (Li, 2025). *Sello* ("Notarized Agents") inverts the trust boundary: the service that receives an agent's call signs a receipt, encrypts it to the agent's owner, and publishes it to a witness-cosigned Merkle log, so that the owner need not trust the agent or its operator (Figuera, 2026). *Janus* places a signed, hash-chained log on the effect path of agentic LLMs, so that a step's proposal and verdict are durable before its effect is released. Its gates are pure functions of the log and can be re-derived offline, and it is evaluated under crash injection and with a lending workflow (Arslan, 2026). Two further systems are run in the comparison of Section 6.7 but are not tabulated here: Agent Flight Recorder, which keeps tamper-evident audit trails of tool-using agents with on-chain anchoring (Bindschaedler et al., 2026), and NovaFabric, which seals tamper-evident, replayable evidence for autonomous agent runs (Seyedkazemi Ardebili, 2026).

Table 1 compares these systems with the artifact presented here. The comparison shows that the contribution of this study is not the idea of signed records for AI, which all four share, but a specific combination of design features aimed at *organizational* model use: issuer-side receipts for single calls and crews that have a signer, a deterministic routing decision whose selected route is signed, redaction before signing, and signed output binding. The comparison also shows what Lattice lacks: Sello's receiver-side attestation and transparency log, Janus's effect gating and crash-consistency evaluation, and a commitment to the task text and the policy, which Lattice receipts do not carry (Section 5). Lattice was designed independently. Receipt code first appears in the project repository (Turlapati, 2026) in commit `fa6d5a1` of 11 May 2026, before the Sello (June 2026) and Janus (September 2026) preprints appeared. Audience, nonce, and expiry binding, chain verification, and chain closure were added in October 2026, after both.

Table 1: Comparison of run-level and agent-level accountability approaches (based on each system's published description; Section 6.7 reports what was measured)

Feature	TrustTrack	Sello	Janus	Lattice (this study)
Unit of record	Agent behavior	Agent action at receiving service	Agent step (proposal, verdict, effect)	Model run (input-artifact and output hashes, contract hash, selected route)
Who signs	Protocol participants (design)	Receiving service	Agent runtime, validators, humans	Issuing runtime (self-attested)
Tamper evidence	Proposed	Signature and witness-cosigned Merkle log	Signed hash chain	Signature over canonical JSON; parent links; crew chains list their members
Third-party transparency log	Not specified	Yes	No	No
Deterministic, recorded routing across providers	No	No	No	Yes; the selected route is signed, the candidates and policy are not
Redaction before signing	Not specified	Encryption to owner	Not a focus	Yes, masked before signing (Sect. 6.5)
Signed output binding	Not specified	Signed receipt over the action	Signed hash chain; verdicts re-derived offline	Signed output hash, checked by re-hashing the kept output
Effect gating	No	No	Yes	No; output invariants run after generation and fail the run
Implementation	None (blueprint)	Reference implementation	Implementation	Open-source library and command-line tool
Evaluation	Use cases	Cryptographic microbenchmarks	Crash injection, 100M-event verification, model experiment	Tamper suite, output binding, policy checks, overhead (Sect. 6)

3 Research Approach

We follow the design science research methodology (DSRM) of Peffers et al. (2007), guided by the DSR guidelines of Hevner et al. (2004) and the contribution framework of Gregor and Hevner (2013). Table 2 maps the six DSRM activities to this study. The entry point is problem-centered: the accountability gap arose while building a browser-automation product whose autonomous actions required an auditable record, and it generalizes to any organization that embeds models in business processes. DSR builds and evaluates artifacts to solve organizational problems while producing prescriptive knowledge (vom Brocke et al., 2020), and it has long been linked to information systems design theories (Walls et al., 1992). We use the anatomy of a design theory (Gregor and Jones, 2007) to structure the knowledge contribution, with requirements as meta-requirements, principles as principles of form and function, and Lattice as an expository instantiation. Evaluation follows the Framework for Evaluation in Design Science (FEDS) of Venable et al. (2016). We adopt a technical risk and efficacy strategy, starting with artificial, summative evaluation of the instantiation against its requirements, and plan a naturalistic evaluation with practitioners as the next episode (Section 6.9).

Table 2: Mapping of DSRM activities (Peffers et al., 2007) to this study

DSRM activity	This study
1. Problem identification	Accountability gap between governance expectations and model-run records (Sect. 1, 2)
2. Objectives of a solution	Design requirements DR1–DR5 (Sect. 4)
3. Design and development	Design principles DP1–DP4 instantiated in Lattice (Sect. 5)
4. Demonstration	Runnable scenarios: a support work inbox whose message, call transcript, policy, and photo inputs are text fixtures; a single-agent loop; and a multi-agent crew
5. Evaluation	Offline, artificial evaluation against DR1–DR5; planned practitioner study (Sect. 6)
6. Communication	This article; open-source artifact and evaluation harness

Evaluation criteria are drawn from the taxonomy of Prat et al. (2015) (goal attainment, efficiency, robustness, and consistency) and from the build-evaluate pattern of Sonnenberg and vom Brocke (2012).

4 Design Requirements and Principles

4.1 Design requirements

We derive five design requirements (DRs) from the governance, auditing, and secure logging literature. Each is stated at the level of the class of systems rather than of any particular implementation.

DR1 (Integrity and attribution). The record of each model run must be tamper-evident and attributable to an accountable issuer, so that any alteration is detectable by a party holding only public verification material. This follows from logging duties for high-risk systems (European Parliament and Council of the European Union, 2024), from the verifiable-claims agenda (Brundage et al., 2020), and from secure audit log research (Schneier and Kelsey, 1999; Crosby and Wallach, 2009).

DR2 (Reproducibility). An auditor must be able to check offline, without access to the original system or provider, that the stored inputs and outputs of a run are the ones the record commits to. This reflects the role of reproducibility in audits (Raji et al., 2020) and in ML practice more broadly (Kapoor and Narayanan, 2023).

DR3 (Inspectable decisions and enforced policy). The record must state which provider and model served the run, and enough of the request, the policy, and the provider catalog must be kept for an auditor to recompute that choice. Declared policies (for example, privacy class, provider restrictions, budget, or citation and personal-data constraints) must be enforced, with violations recorded rather than silently retried. This reflects the inscrutability concern in AI management (Berente et al., 2021), the need for explanations tailored to auditors (Felzmann et al., 2019), and risk-management functions in Tabassi (2023).

DR4 (Data minimization). The record must not disclose secrets or personal data beyond what accountability requires, so that it can be shared with auditors without creating a new privacy risk. This reflects data protection obligations and the transparency–privacy tension discussed by Felzmann et al. (2019).

DR5 (Low adoption cost). Producing the record must add negligible runtime overhead and require little change to existing application code, because accountability mechanisms that impose high costs are bypassed in practice (Sculley et al., 2015; Amershi et al., 2019).

4.2 Design principles

We formulate four design principles (DPs), following the convention that a principle names the action, the mechanism, and the rationale (Gregor and Hevner, 2013).

DP1 (Receipt by default). Provide the runtime with a signing key so that *every* run, whether successful or failed, emits a receipt that is canonicalized, signed, and verifiable with a public key, because accountability evidence that depends on developer discipline is incomplete. (Addresses DR1, DR5.) Lattice realizes DP1 for single calls (`ai.run`) and for multi-agent crews (`runAgentCrew`), but not for its single-agent loop (Table 3).

DP2 (Commit to content, carry hashes). Have the receipt commit to content-addressed hashes of input artifacts and outputs rather than to the content itself, and keep the content in a separate artifact store, because this makes receipts small and shareable while still allowing stored content to be checked against the record. (Addresses DR2, DR4, DR5.)

DP3 (Decide deterministically and record the decision). Implement provider and model selection as a pure function of a capability catalog, the request, and the policy, and record the selected route in the receipt, because a decision that can be recomputed can be audited. Treat output-policy violations as terminal, so that a run is never retried until a banned output passes. (Addresses DR3.)

DP4 (Redact before signing, verify before loading). Apply redaction before canonicalization and signing, so that matched sensitive values never enter the signed bytes in clear form, and verify a receipt before loading any referenced artifact, so that a forged receipt cannot trigger side effects. (Addresses DR1, DR2, DR4.) A hash in the signed bytes hides a value only when that value is hard to guess. A low-entropy value, such as a social security number or a yes/no output, can be confirmed from its unsalted hash by dictionary search. Salted or keyed commitments would close this gap and are left to future work.

5 The Artifact: Lattice

Lattice is an open-source TypeScript library and command-line tool (Turlapati, 2026). A developer describes a run by its task, artifacts (text, structured data, images, audio, documents, or tool results), declared outputs with schemas, a policy, and an optional contract of output invariants. Figure 1 summarizes the run life cycle, and Table 3 maps principles to mechanisms.

Receipt contents. A signed receipt body holds the run identifier, the issue time, the signing key identifier, the requested and observed model, the selected route (provider, capability, and attempt number), token usage and cost, the contract verdict, the hash of the contract, the SHA-256 hashes of the input artifacts and of the output, a Merkle root over artifact lineage, and the redaction manifest. When a tripwire fires, the body also holds redacted tripwire evidence, and when no route exists, it lists the rejection reasons. Optional fields bind an audience, a nonce, an expiry, a parent receipt, and chain closure. The body neither contains nor hashes the task text, the packed prompt, or the policy, and it records the selected route without the candidates, scores, or catalog version that produced it. Two runs that differ only in their task text, and that return the same output with the same token usage, therefore yield receipts that differ only in identifiers and time. DR2 and DR3 hold for what the receipt commits to, not for these omitted inputs.

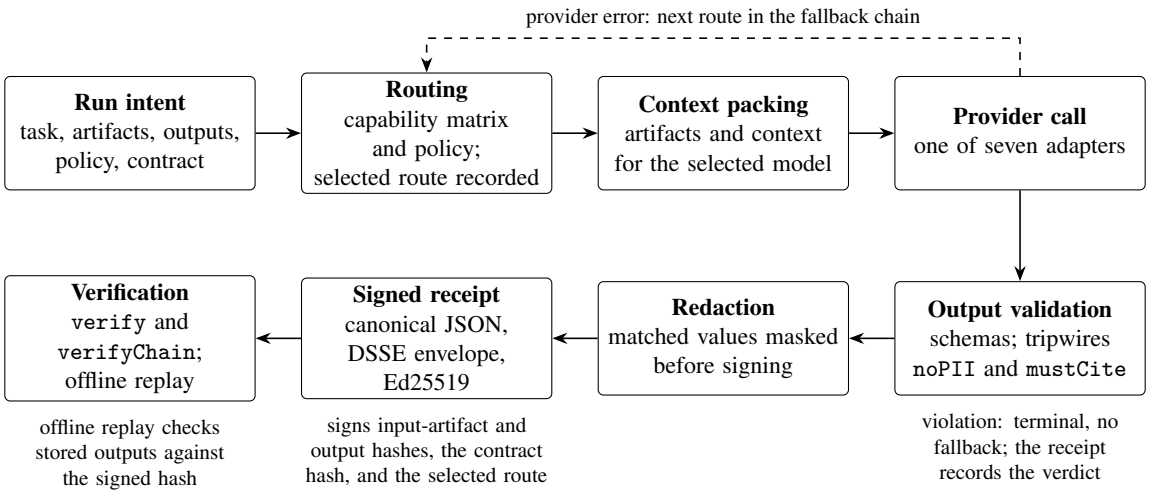


Figure 1: Run life cycle of an `ai.run` call in Lattice. Solid arrows follow one run. The dashed arrow is the fallback taken on a provider error. An output-invariant violation ends the run instead. The task text, the packed prompt, and the policy are not part of the signed receipt

Provider coverage. Seven provider adapters (OpenAI, generic OpenAI-compatible endpoints, Anthropic, Google Gemini, xAI, OpenRouter, and LM Studio for local models) implement a common contract, so a receipt has the same fields whichever provider served the run. What reaches the provider differs. The Anthropic and Gemini adapters send image artifacts as native image parts. The OpenAI, OpenAI-compatible, xAI, OpenRouter, and LM Studio adapters send every artifact as a text part holding JSON, and include the artifact's value only when it is a string. A data-URL string is inlined as text, and other values, such as binary image bytes and JSON objects, are dropped from the request, yet the receipt still commits to the hash of the dropped input. On those five adapters a receipt can therefore list an input the model never saw. The vision probe in Section 6.8 shows the effect. The router consults a capability catalog generated from a public model registry, supplemented by hand-maintained profiles.

Agent runs. With a signer, `runAgentCrew` returns a crew-start root, a completion receipt for the root agent and for each child run that succeeded, and a crew-close receipt, so a crew of N agents that each run once returns $N+2$ receipts when every child run succeeds. A child whose run fails gets no completion receipt. Each receipt after the root links to the root by its content identifier. The crew-start output hash commits to the nested tree of agent identifiers. It does not cover the agents' intents, tools, or policies. Completion receipts hash each agent's output. The root declares `chainClosure`, and the crew-close receipt hashes and lists the content identifiers of every other member, so `verifyChain` rejects a closed chain with a dropped leaf or a dropped closer. Verifiers in the 1.6.x line accept these receipts but do not enforce closure. Later verifiers enforce it. Two verifier options extend the check: `requireClosure` fails a chain that declares no closure, and `partial` accepts a disclosed subset only when the closer is present and lists every presented receipt. The crew budget verdict is fixed when agent work ends, before the root agent's completion receipt and the crew-close receipt are signed, so signing those two receipts cannot push a crew over budget, and the crew-close receipt signs a failed verdict when the crew exceeded its budget. The crew-start and child completion receipts are

Table 3: Design principles and their instantiation in Lattice

Principle	Mechanism in Lattice
DP1 Receipt by default	A signer configured once at initialization causes a receipt on every terminal outcome of `ai.run`. `runAgentCrew` returns a crew-start receipt, a completion receipt for the root agent and for each child run that succeeded, and a crew-close receipt. The single-agent loop `ai.runAgent` returns no receipt; with a signer on its intent, its per-iteration checkpoint receipts go only to a configured tracer. Receipts use RFC 8785 JSON canonicalization (Rundgren et al., 2020), a DSSE envelope (Secure Systems Lab, 2021), and Ed25519 signatures (Josefsson and Liusvaara, 2017; Bernstein et al., 2012). A key set records each key as active, retired, or revoked, and an explicit schema version blocks downgrade to the original unversioned schema.
DP2 Commit to content	Receipts contain SHA-256 hashes of each input artifact, a Merkle root over artifact lineage (Merkle, 1988), and a hash of the outputs; artifacts are stored by content address. Monetary cost is serialized as an I-JSON string (Bray, 2015).
DP3 Deterministic decision	The router applies hard filters (modality, context window, structured output) and policy filters (allow and deny lists, privacy class, logging, latency, budget), then ranks survivors by a score combining cost, latency, and context headroom. Ties break on provider and model identifiers, not on catalog position. Output invariants (`mustCite`, `noPII`, `fieldFromTable`, `matches`) are evaluated by a pure function; violations are terminal and excluded from fallback.
DP4 Redact, then sign; verify, then load	Canonicalization is called only on the output of redaction. Materialization for offline replay verifies the receipt before any artifact is loaded. Input and output hashes are unsalted SHA-256.

signed during the run and count against its wall time. If a mint after the crew-start receipt fails, the result lists every failed mint in `receiptError` by default, and the crew throws at the failed mint when `onSignerError` is set to throw. A missing closer leaves the closed chain unverifiable. The single-agent loop `ai.runAgent` returns no receipt (Table 3).

External executors. An organization that already uses another orchestration framework can call an audit helper that mints the same receipt family from externally produced request, response, and output data. This supports incremental adoption (DR5), at the price of trusting the external executor's account of the provider call.

Command-line tool. The tool verifies receipts and receipt chains, prints structured advisories as warnings or information lines, checks stored artifacts and outputs offline against the signed hashes, compares receipts structurally, and gates regressions in cost and output consistency across a corpus of receipts in continuous integration. The library can also export run events through OpenTelemetry (OpenTelemetry Authors, 2026), and tool results from the Model Context Protocol (Model Context Protocol, 2025) can be recorded as artifacts.

6 Evaluation

6.1 Evaluation design

We conducted an artificial, summative evaluation (Venable et al., 2016) of the instantiation against DR1–DR5. The harness is published with the artifact (`eval/run-all.sh`). In that artificial evaluation, provider calls were served by in-process fakes that return provider-shaped responses, so no network, credentials, or

Table 4: Evaluation of design requirements: experiments, criteria (Prat et al., 2015), and outcomes

Requirement	Experiment	Criterion	Outcome
DR1 Integrity	Tamper suite on 70 receipts; chain splicing; verifier opt-ins	Robustness	Single-receipt attacks 3570/3570; freshness binding and open-chain length are opt-in; closed crew chains fail on a missing member
DR2 Reproducibility	Record, persist, reload, and re-check 350 runs across 7 adapters	Consistency	Hash match 350/350; binds artifacts and outputs, not the task text; does not cover live model behavior
DR3 Decisions and policy	1000 routing requests with an independent oracle; labeled invariants	Goal attainment	No policy violations (0); permutation-stable (1.0000); selected route signed, rationale not; PII and must-cite below
DR4 Minimization	Secret-leakage scan of signed receipts	Goal attainment	Marker secrets in signed bytes: 0 of 14 rows; low-entropy hashes can be guessed
DR5 Low cost	Latency over 1000 trials; receipt sizes	Efficiency	Signing delta 0.188 ms (small text) to 1.447 ms (1 MB image), p50

paid services were involved. Table 4 maps requirements to experiments, and Figures 2, 3, and 4 plot the personal-data check, the per-run latency, and the cost of receipt chains. Section 6.7 is separate: it records a local open-weight model and the other record systems that could be executed on the same machine (Table 5; Figures 5, 6, and 7). Section 6.8 adds free hosted models (Tables 6 and 7), public personal-data datasets (Tables 8 and 9; Figures 8 and 9), a citation dataset, and local gateways. Its other figures give Lattice processing share, signed latency, answer drift, and output binding on the live models (Figures 10, 11, 12, and 13), mutation results and observed capabilities across the record systems (Figures 14 and 15), and the gateway runs (Table 10; Figure 16). Synthetic data were generated with fixed random seeds. Hardware, the Node version, and the git commit are in Appendix A and in the environment files under `eval/results`, not in this narrative, because they are properties of the measurement machine. Tables in the appendix are generated from the JSON the harness writes: latency, receipt size, and chain cost (Tables 11, 12, and 13), and the replay, policy, tamper, and verifier checks (Tables 14, 15, 16, and 17). The prose below quotes those generated values through macros, so a number in the text is the number in the file.

6.2 Integrity under attack (DR1)

We generated 70 receipts from runs across all seven adapters and applied the mutations in Table 16. Of the cases the harness classifies as attacks, 3570 of 3570 were rejected. Body edits failed signature verification whether the attacker kept the original signature, re-canonicalized correctly, or re-signed with an untrusted key under the legitimate key identifier. Non-canonical encodings were rejected as canonicalization mismatches. A valid receipt presented a second time is accepted: the verifier keeps no replay cache. Audience, nonce, and a caller-supplied version floor are checked only when requested. `expiresAt`, when the issuer set it, is enforced on every verification. Presenting the same receipt again is therefore not, by itself, a failed verification. What the verifier did not check is reported as structured advisories, each with a code and a severity. A warning means the receipt is bound to an audience or a nonce that the caller did not check. An information advisory means the receipt carries no audience, nonce, or expiry, or sits below the current schema version. The command-line verifier always prints warnings and prints information lines unless `-quiet` is set.

Figure 14 applies comparable mutation classes to the other record systems of Section 6.7.

Chain checks are separate from the ten `VerifyErrorKind` values. `verifyChain` reports its own error kinds (`VerifyChainErrorKind`): an invalid link, no root, multiple roots, an orphan link, a run-identifier mismatch, a duplicate content identifier, a count mismatch, or a root mismatch. Reordering the receipt array is not an error, because the verifier follows parent links rather than array position. Dropping a middle link, splicing a foreign link, or replacing the root fails that check. Truncating the tail of an open chain does not. The result is success plus a tail-truncation warning unless the caller passes `expectedCount`. A chain whose root declares `chainClosure` fails with a count mismatch when a leaf or the closing receipt is missing, with no expected count, and a malformed closing receipt fails as an invalid link. `requireClosure` turns a missing closure declaration into a failure, and `partial` accepts a disclosed subset only when the closing receipt is present and lists every presented receipt, and warns when some listed members were not presented. Table 17 records these checks together with the advisories, crew receipts, and signer-failure modes. 71 of 71 checks passed. Two of them run the released 1.6.0 verifier on a closed crew chain and confirm that it accepts the chain without enforcing closure.

6.3 Reproducibility (DR2)

For each of the seven adapters we recorded runs against provider-shaped fixtures, wrote the receipt and content-addressed artifacts to disk, reloaded them, verified the receipt, materialized a replay envelope, replayed offline, and recomputed the output hash. 350 of 350 runs verified, and 350 reproduced the signed output hash. Re-executing each run against the same fixture matched in 350 cases, and a one-character change to a recorded output was detected in 350 cases (Table 14). These results show that the record is internally consistent. Offline replay does not call a model: `replayOffline` returns the outputs the caller supplies, and the check is that they hash to the signed output hash. The result is therefore a signed output binding, which other signed record systems also provide (Table 5). The same binding held on every live call that returned a receipt (Figure 13). It does not show that a live model would reproduce its output, and repeated live calls often returned different answers (Figure 12). The binding covers input artifacts and outputs only. Because the task text is not committed, a receipt does not show which instruction produced the output. In a crew chain, the crew-start receipt hashes the nested agent tree (`{kind: "crew-start", agents: {id, children}}`), each completion receipt hashes its agent's output, and the crew-close receipt hashes its member list.

6.4 Inspectable decisions and policy enforcement (DR3)

Routing. Using a synthetic catalog of 40 model capabilities and 1000 random requests with random policies, repeated decisions were identical at rate 1.0000. An independently implemented oracle found 0 selected routes that violated provider allow or deny lists, privacy class, logging, latency, or image-modality constraints, and 0 cases in which the router reported no route although a feasible candidate existed. Reordering the catalog changed the selected model at a same-selection rate of 1.0000. Catalog position is not part of the score, and ties break on provider and model identifiers. The receipt signs the selected route and the requested and observed model. It does not sign the policy, the candidate list, the scores, or the catalog version,

so an auditor who wants to recompute the decision needs those from records kept outside the receipt.

Personal-data invariant. The `noPII` detectors cover four types, and each rule trades recall for precision in a stated way. Email addresses match as written. A dashed United States social security number (SSN) matches as written. Spaced and undashed nine-digit forms match only when an SSN cue such as `ssn`, `ss#`, or `social security` is the nearest word. Separated United States phone numbers match as written. A plain ten-digit run matches only when it is a valid North American number and the nearest word is a phone cue (`phone`, `tel`, `call`, `mobile`, `cell`, or `fax`) or a `+1` or `1-` trunk prefix precedes it. A payment card number is any Luhn-valid run of 13 to 19 digits, except runs that start with 0 or 1 and 13-digit runs that start with 978 or 979, which are ISBN-13 numbers. An SSN, phone, or card match is suppressed when an identifier label (`order`, `ord`, `invoice`, `inv`, `sku`, `ref`, `po`, `isbn`, `pn`, or `txn`) is the nearest token on the same line, joined to the number only by whitespace or by `:`, `#`, `=`, `no.`, `number`, or `is`, unless the number is a phone number with a trunk prefix. Labels do not suppress email addresses. Words such as customer, account, and ticket are not labels. The trade-offs follow from these rules. A real SSN written as `order 123-45-6789` is not flagged. A bare nine- or ten-digit number without a type cue is not flagged, whether or not it is personal data. About one in ten random digit strings passes the Luhn check, so a text that lists k numeric identifiers of 13 to 19 digits is flagged as holding a card with probability near $1 - 0.9^k$. The detectors see only strings, so a value stored under a key such as `ssn` is judged by its digits alone.

We built a labeled set of synthetic strings, 100 per label. In-scope positives cover each form the detectors claim, support-ticket text in which a number follows words such as customer, account holder, or patient ID, CSV and markdown rows, and card numbers from several networks. Out-of-scope personal data are names with addresses, international bank account numbers, United Kingdom phone numbers, IP addresses, dates of birth, and spelled-out email addresses. Negatives include order identifiers and stock-keeping units shaped like SSNs or phone numbers, 16-digit references, ISBN-13 numbers, and uncued numbers such as timestamps, byte counts, and bank routing numbers. On in-scope types plus all negatives, `noPII` reached precision 1 and recall 0.9429 ($n = 5500$, false positives 0, false negatives 200). The false negatives come from forms the rules leave out by design, such as a plain ten-digit number after "call the customer at" and an undashed SSN in a CSV row with no SSN cue. The 1000 uncued numeric negatives produced 0 false positives. Counting every personal-data label, including types the detectors do not claim, precision was 1 and recall 0.8083 (Table 15, Figure 2). The cases were chosen with the rules in view, and the identifier-shaped negatives carry the labels the rules key on, so these figures show that the rules behave as specified. They are not prevalence-weighted estimates and not estimates of precision on real text. On public datasets, SSN and phone precision and recall were far lower, because passport numbers and other identifiers share the same digit shapes (Section 6.7, Figure 6, Tables 8 and 9, Figures 8 and 9). Those public-data rows were measured with the rules at commit `a8cc091`, which differ from the rules above as Section 6.7 describes.

Citation invariant. The `mustCite` invariant checks that an output cites a named artifact. By default it reads only structured lists: a top-level `citations` or `evidence` list whose string entries, or whose objects' `source` field, name the artifact. Matching is case-sensitive unless case folding is requested, and other object keys can be named. If the top-level list is missing or omits the artifact, nested `citations` and `evidence` lists are

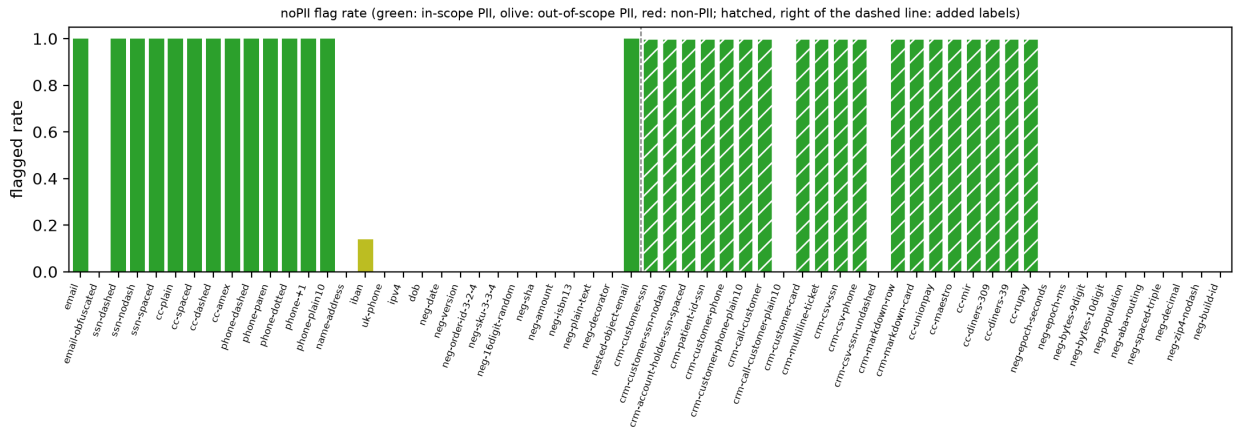


Figure 2: Share of samples flagged by the `noPII` invariant, by label. Green: in-scope personal data. Olive: out-of-scope personal data. Red: non-personal data. Plotted from `eval/results/policy.json`

searched, unless the contract declares a path. The default never reads prose. Three options extend it, and each is added to the contract only when set, so the hash of an existing contract does not change. With `inline`, a bracketed or parenthesized group in the prose whose content, or one comma- or semicolon-separated part of it, equals the artifact name also counts; it does not override a non-empty structured list that omits the artifact. With `rejectUncited`, a matching list entry is rejected when the prose contains citation-like square brackets, such as `[1]`, `[1-3]`, or `[Smith 1999]`, and none of them resolves to the artifact. Links, footnote markers, and code are not treated as citations. `textPath` limits these prose checks to one field. The invariant agreed with the labels in 36 of 36 constructed cases: 22 of 22 for the default and 14 of 14 for the options. The cases include numeric markers, links, and path-scoped outputs, and they were also chosen with the implementation in view. ASQA long answers are prose without citation lists, so the default accepts a gold title in none of them. The ASQA count in Section 6.7 was measured at commit `a8cc091`, where an inline marker containing the name counted by default. The check is syntactic. It does not judge whether the cited artifact supports the claim (Lewis et al., 2020).

6.5 Data minimization (DR4)

We placed marker secrets in the task, an input artifact, the provider credential, and the model output, then scanned the decoded signed payload, with and without a `noPII` invariant. Marker hits in signed bytes were 0 of 14 rows (Table 15 does not repeat the per-adapter log; the raw rows are in `eval/results/policy.json`). Receipts carry hashes of the input artifacts and the output. The task text and the credential appear in the signed bytes neither in clear nor as a hash. When the invariant fires, the in-process tripwire result still includes the matched substring so the caller can act on it. The value that is signed is a detector name, a length, and a mask, so the signed bytes hold the masked form that the redaction manifest describes. The scan checks for the marker strings themselves. It does not test whether a hash can be inverted: a low-entropy input or output, such as an SSN or a yes/no answer, can be confirmed from its unsalted hash by dictionary search (Section 4, DP4). Among the record systems compared in Section 6.7, Figure 7 shows which kept the

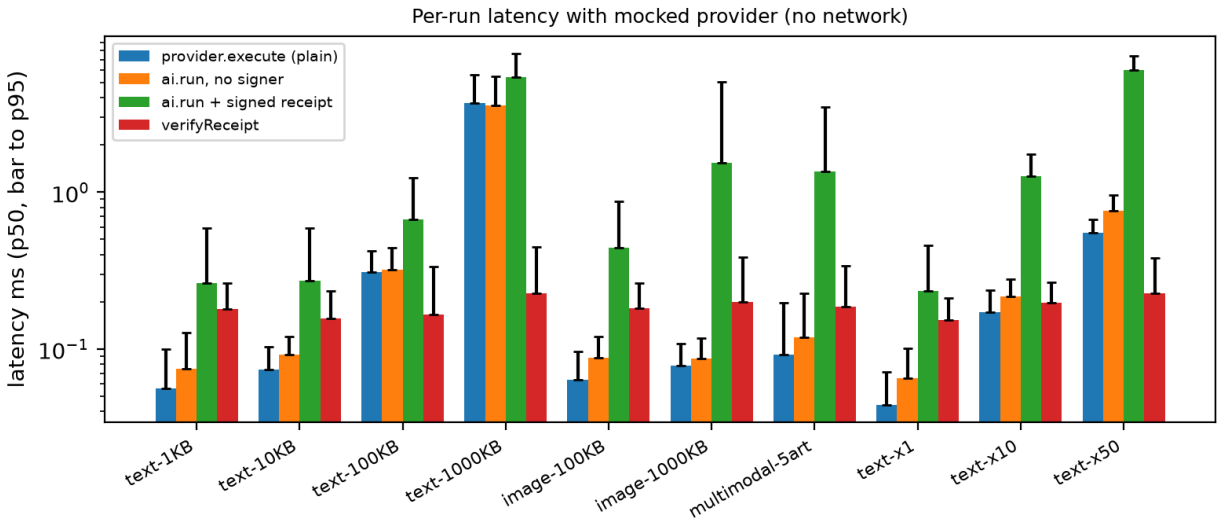


Figure 3: Per-run latency with a mocked provider across input configurations (log scale; bars show the median, whiskers extend to the 95th percentile)

plaintext transcript and which kept only digests or ciphertext when wrapping the same recorded request and response bytes.

6.6 Adoption cost (DR5)

Table 11 and Figure 3 report latency over 1000 trials after warm-up, against a mocked provider. Signing added 0.188 ms at the median for a 1 KB text input and 1.447 ms for a 1 MB image (signed run minus unsigned run). Verification of the small-text receipt was 0.180 ms at the median. The receipt grows with the number of input hashes rather than with payload size (Table 12). Linked receipts and chain verification scale with depth (Table 13, Figure 4). Because the provider was a zero-latency fake, these figures are the runtime's own cost. Sections 6.7 and 6.8 measure that processing time again on a local model and on hosted models, with provider latency and rate-limit waits kept on separate clocks (Figure 5, Tables 6 and 7, Figures 10 and 11).

6.7 Local open-weight runs and other record systems

The artificial evaluation mocks the provider. This subsection does not. Ollama 0.40.7 served `llama3.2:3b` (Llama 3.2, 3.2B, Q4_K_M, digest `a80c4f17acd5`, 2.0 GB) on the same host as the other measurements (Node v24.22.1, 4 vCPU, 15.63 GB RAM, no GPU). Lattice called it through the OpenAI-compatible adapter with no API key and an explicit base URL ending in `/v1`. `createOpenAIProvider`, whose default base URL builds a chat URL without `/v1`, was not exercised. The adapter does not send temperature or a token cap, so a fetch wrapper added both to every request. The Lattice numbers are therefore for a request the adapter does not build on its own. No vision model was loaded. No hosted provider key was present.

On the one-line prompt "Reply with the single word pong," temperature 0, five paired trials, the median

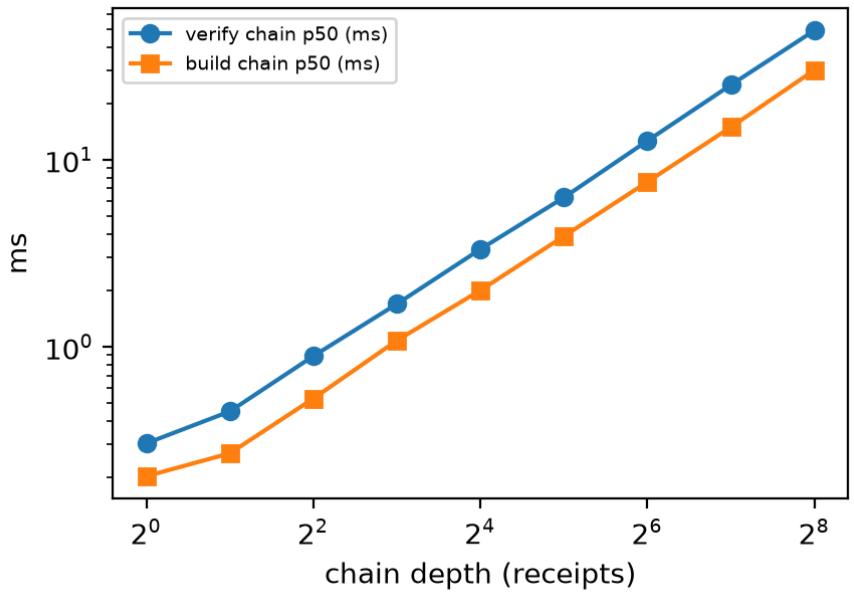


Figure 4: Build and verification time of linked receipt chains by depth

plain HTTP call was 278 ms. A signed Lattice run of the same task was 559 ms and an unsigned Lattice run was 745 ms (Figure 5). The first plain call, 4845 ms, is a cold start and pulls the mean up; the median does not. The signed median is lower than the unsigned median. That gap is generation noise, not a claim that signing is faster. What is stable is the time spent outside the provider HTTP call: 0.439% of end-to-end latency at the signed median (minimum 0.335%, maximum 2.469%). The Lattice prompt body was 382 characters against a one-line plain prompt, so most of the extra wall time is a larger prefill, not the signature. The short signed receipt was 996 bytes.

A refund-policy question with a `mustCite` invariant passed on all three temperature-0 runs and all three temperature-0.7 runs. These verdicts come from the `mustCite` rules at commit `a8cc091`, where an inline `[name]` or `(name)` marker containing the artifact name counted as a citation by default. The current default reads only a structured citation list, so these answers, which cite the clause only inline, would fail it unless `inline: true` is passed. Each temperature produced one identical answer across its three trials (n=3). The two temperatures did not produce the same sentence. All 6 signed receipts verified. For each, the signed output hash matched the kept answer and changed after a one-character edit (6/6). This checks the output binding, not a re-run: `replayOffline` returns the outputs the caller supplies. A synthetic support ticket containing an email, a US phone number, and a dashed SSN was flagged by those three detectors. The model's answer contained none of them, the `noPII` invariant passed, and the signed receipt did not contain the planted strings. A two-step agent called `lookup_policy` once and answered; the returned agent object carried no receipt. A two-agent crew completed in 3 iterations with 4 receipts.

The public sets are harder than the synthetic harness, and the gap is the result (Figure 6). ai4privacy/pii-masking-300k (Ai4Privacy, 2024) English validation (revision `c8c77895a005`, 7,946 documents; academic, non-commercial, no redistribution) gives email precision 0.955 and recall 0.979 (1533 documents with a gold

email). US SSN precision is 0.325 and recall 0.436. US phone precision is 0.316 and recall 0.333. Of the SSN false positives, 1,074 overlap a passport span: a 9-digit passport is the same shape as an SSN, and the detector cannot tell them apart. Of the phone false positives, 629 overlap a social-security span. There is no gold credit-card span in this file; the card detector fired on 7 documents anyway. Among 1,070 US-shaped SSN spans, the single match the API returns overlaps 663. It misses 370 because that one match fell on a different number earlier in the document, and 37 because it matched nothing. Gretel English finance test (Gretel.ai, 2024) (Apache-2.0, revision `7b844d167385`, 2,891 rows) is the same pattern at smaller n: email recall 0.934, SSN precision 0.161. ASQA dev (Stelmakh et al., 2022) (Apache-2.0, revision `084060f16b46`, 948 answers) is a loss for `mustCite`. A gold Wikipedia title occurs as a substring in 525 answers, but the invariant accepts at least one gold title in 7. Only 2 answers use a square-bracket citation. The `mustCite` count was measured at commit `a8cc091`, where an inline `[name]` or `(name)` marker containing the title counted as a citation by default. The current default never reads the answer text, and these answers carry no citation list, so it accepts a gold title in none of them. The opt-in `inline: true` counts only a bracket or parenthesis whose content, or one comma- or semicolon-separated part of it, equals the title. The check is syntactic. It does not score whether a citation supports the claim. These dataset rows were measured with the `noPII` detectors at commit `a8cc091`. The current detectors differ: an identifier label suppresses a match only when it is the nearest token, and bare 9-digit, spaced, and 10-digit numbers need a type-specific cue such as SSN or phone. The artificial evaluation covers the current detectors and the current `mustCite` rules.

Table 5 wraps the same recorded request and response bytes. Lattice's receipt and Sello's encrypted receipt (mock transparency log, not the public log) omit the planted strings. An AFR log with disclosure encryption also omits them; the default AFR log contains them. Janus's local evidence appender stores the payload in the clear and the byte edit fails `verify.SegmentDir`. A sealed NovaFabric capsule (self-asserted ECDSA key, no RFC 3161 timestamp) fails `nova verify` after a newline is appended to stdout, and the capsule contains the planted text. `nova validate` on that edited capsule still exits 0. cosign `sign-blob` (The Sigstore Authors, 2021) with a local key and no Fulcio certificate detects a byte edit and signs the plaintext log. That run kept cosign's default transparency-log upload, so it wrote an entry to the public Rekor log. Its 4,916 ms wrap time includes that round trip, and `verify-blob` checked the log online. in-toto's DSSE link (Torres-Arias et al., 2019) detects a byte edit and does not embed the plaintext. A hash chain detects an edit that leaves the stored hashes untouched and accepts a rebuilt chain, because nothing outside the file anchors the head. NovaFabric's HMAC record has the same shape with a shared secret: an edit fails, and a re-sign with that secret passes. OpenTelemetry GenAI spans and a plain JSON log have no integrity check; both store the transcript. TrustTrack has no implementation we could run. Langfuse was not started: this host has no Docker daemon. Lattice, the NovaFabric capsule, cosign, in-toto, Sello, and Janus sign the record and reject a byte edit; Table 5 lists that as signed output binding.

The wrap times are single cold samples with different scopes, so they order the systems only roughly. The Lattice total of 5.923 ms is three `createReceipt` calls, and the first call is 5.584 ms of it; Sello's first call is 7.259 of its 9.336 ms. The byte column mixes scopes too (Figure 7): the JSON log, the hash chain, OTel spans, AFR, the NovaFabric HMAC record, the NovaFabric capsule, cosign, and Janus keep the plaintext transcript, while Lattice, AFR disclosure, in-toto, and Sello keep only digests or ciphertext.

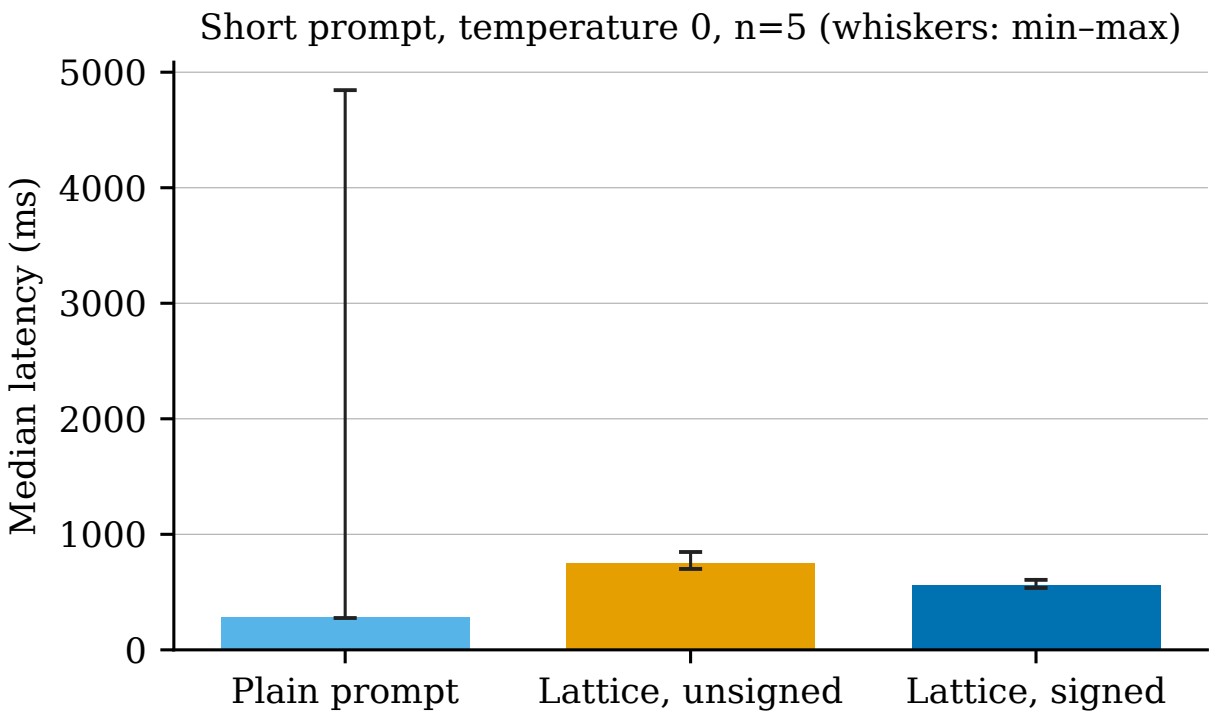


Figure 5: Median latency of a one-line completion. Whiskers run from the minimum to the maximum of five trials. The plain arm sends only that line. The Lattice arms send the adapter's packed prompt.

6.8 Broader comparison

This subsection adds the detector, citation, gateway, and hosted-model measurements from the same machine.

Lattice's OpenAI-compatible adapter called `https://openrouter.ai/api/v1`. The catalog check found 8 of the requested `:free` ids, and the listed prompt and completion prices were 0. Completed model rows: 8. Upstream HTTP 429 responses were retried. Pacing sleeps, that backoff, and the HTTP time of a failed attempt are excluded from Lattice time and from end-to-end time, so end-to-end time is Lattice processing plus the provider time of the attempt that returned. Provider time includes waiting for the response headers and reading the body. Completion tokens are capped at 8192. The adapter sends no temperature, `max_tokens`, or reasoning field. A fetch wrapper adds temperature and the token cap to every request, and adds `reasoning.effort=low` for models whose catalog lists a reasoning parameter. A 400 that rejects that field is retried once without it. The Lattice rows therefore describe requests the adapter does not build on its own. Table 6 uses the provider string returned on the chat response. A drift cell is distinct successful answers, successful calls, and attempts. A RAG call succeeds when the run returns an answer that passes `mustCite`. The recorded verdicts come from the rules at commit `a8cc091`, where an inline `[name]` or `(name)` marker containing the artifact name counted as a citation by default. The current default reads only a structured citation list and would fail every recorded RAG answer unless `inline: true` is passed. Output binding uses only calls that returned a receipt. The local row is the Ollama run in Section 6.7 (drift n=3, overhead

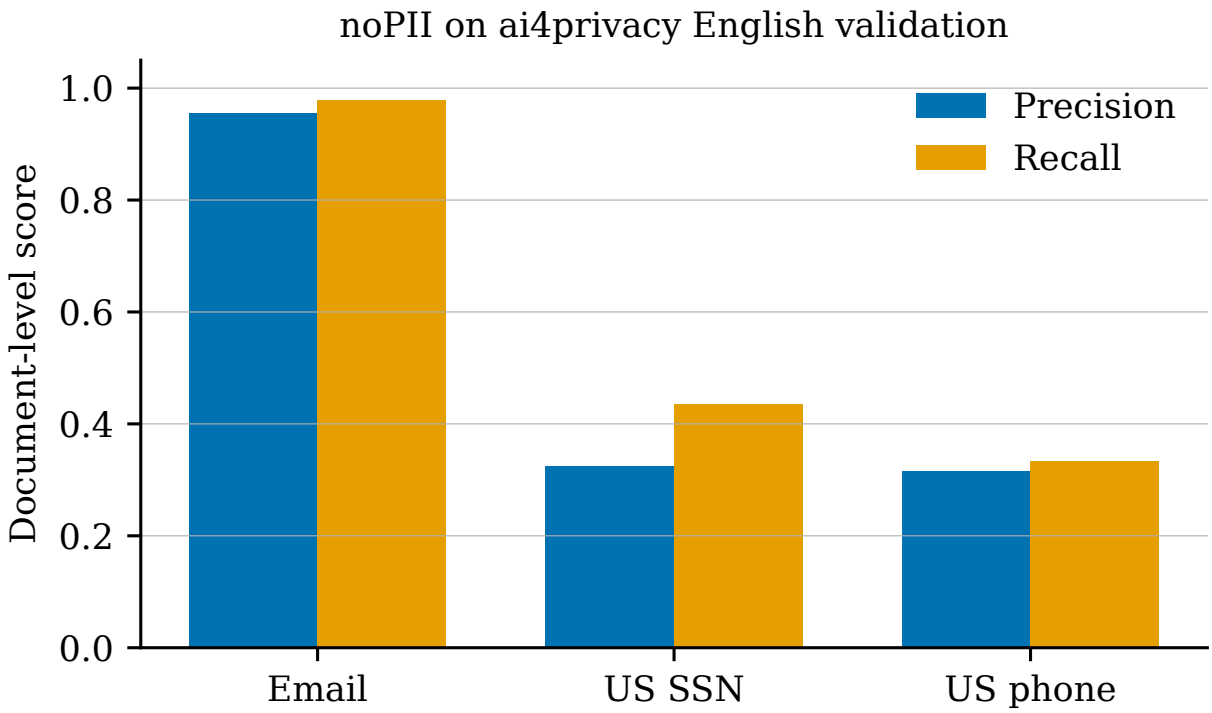


Figure 6: Document-level precision and recall of the default `noPII` detectors at commit `a8cc091` on ai4privacy English validation. US SSN and US phone lose to identifier types that share their shape.

n=5). Hosted drift used n=10 when the call returned. On the local Ollama run, the median time outside the HTTP call was 2.4 ms (p95 15.0 ms, 0.439% of that run's signed wall time). That localhost call returns the body with the headers, so the gap is processing time. Across hosted models with a successful signed short prompt, the median Lattice processing time was 1.10–1.69 ms, or 0.032–0.218% of end-to-end time. Table 7 gives the median and p95 in milliseconds and as a percentage. The percentages are recomputed from the per-trial phase times; the `signedLatticePct` field stored in `openrouter.json` also counts the HTTP time of retried attempts. Lattice time includes `createAI()` construction and the fetch wrapper's JSON handling, so it is an upper bound. The packed Lattice prompt added 61–83 prompt tokens at the median versus the one-line direct prompt. The paired provider-time difference on that longer prompt had medians from -2398 to 3352 ms. A negative value means the packed call returned sooner. The difference changes sign and is often larger than a short extra prefill can explain, because completion length, including reasoning tokens, moves the provider time by seconds. It is reported separately and is not added to Lattice processing time.

`thinkingmachines/inkling-small:free` is in the public catalog at price 0. Every measured call was rejected with HTTP 403 and the message that the model is only available on agentic harnesses, so this row has no latency sample. `liquid/lfm-2.5-2.6b:free` returned HTTP 504 on the single-agent call (Provider returned error). That call is a provider failure, not a receipt or citation result. `liquid/lfm-2.5-2.6b:free` returned HTTP 504 on the crew call (Provider returned error). That call is a provider failure, not a receipt or citation result.

On a 16×16 solid red PNG, `nvidia/nemotron-3.5-super-vl:free` answered red from a direct

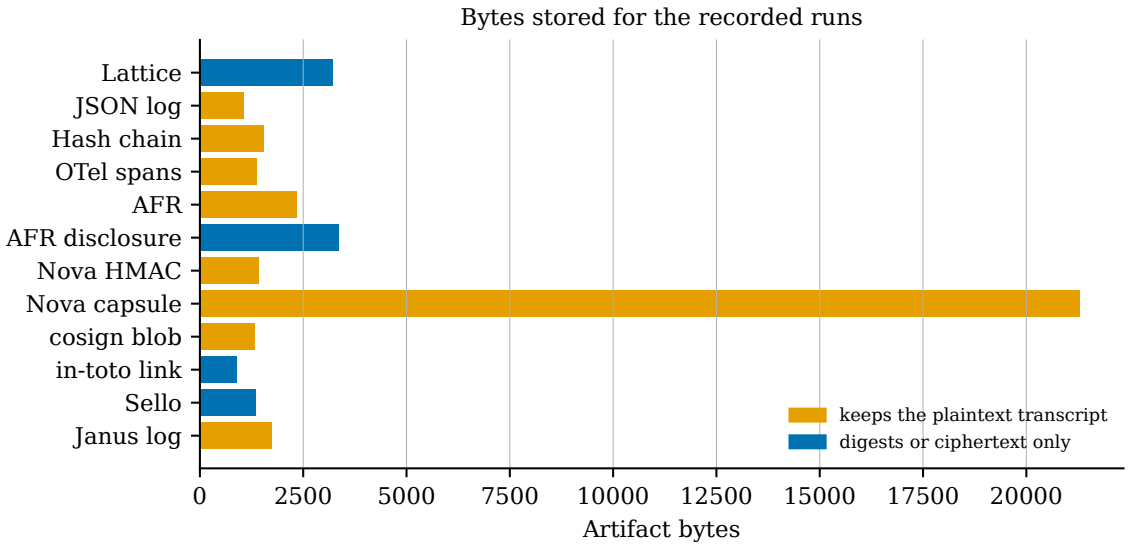


Figure 7: Bytes retained for the recorded runs. Orange records keep the plaintext transcript; blue records keep only digests or ciphertext. The NovaFabric bar is one sealed capsule, not three hash receipts. AFR disclosure is the encrypted archive. The cosign bar is the blob, the signature, and the public key.

`image_url` part: yes (answer red). The Lattice image artifact answered red: no (answer blue). The PNG bytes were present in the Lattice request body: yes. The direct response provider was Nvidia. This vision call set `reasoning.effort=none`. The text arms use reasoning.effort=low. On this vision call, effort low and effort minimal each spent 8192 completion tokens on reasoning and returned empty content. This recorded call sets reasoning.effort=none. Lattice inlines the PNG in the text JSON. The direct arm sends an image_url part. Lattice's OpenAI-compatible request builder, which the openai, xai, openrouter, and lm-studio adapters share, sends each artifact as a text part holding JSON and never emits an image part. A data-URL string is inlined as text and binary bytes are dropped, so the model received no image part, and the Lattice arm answered blue for a red PNG.

Document-level precision, recall, and F1 use the harness overlap rule: a document is a true positive when any predicted span of that type overlaps a gold span. An undefined F1 means there were no positive predictions, or no gold documents and no false positives. spaCy `en_core_web_sm` emits none of the four labels, so its precision and F1 are undefined and its recall is zero wherever a gold span exists. Figure 8 omits those undefined bars.

The rule is not symmetric across tools. For each type, Lattice contributes every occurrence of the one substring `detect()` returns, while the other tools contribute every span they emit. When Lattice's one match falls on a different number, the document is a false positive and a false negative for Lattice, while a multi-span tool can still score it from another span. GLiNER (Zaratiana et al., 2024) is the general `urchade/gliner_small-v2.1` model, not a PII fine-tune, and its processor truncated each document to 384 tokens, while LLM Guard (Protect AI, 2023) scanned 1,500-character windows with 200 characters of overlap. LLM Guard ran its default Anonymize scanner with recognizer `Isotonic/deberta-v3-base_finetuned_ai4privacy_v2`, which was fine-tuned on ai4privacy, so its ai4privacy scores can be optimistic. A single process was

Table 5: Same recorded Ollama runs, wrapped by each system that ran. Wrap time excludes model generation. Wrap ms is one cold run per system, not a warmed median, and the scopes differ: Lattice and Sello are in-process JavaScript calls summed over three records, cosign and the NovaFabric capsule are one CLI invocation each, including process start and key loading, Janus reports whole milliseconds from its own timer, and the JSON log time is serialization only. PII in artifact reflects what the harness gave each system; where it is yes, the byte count includes the plaintext transcript. Signed output binding is yes when the record is signed and the system's own verifier rejected a byte edit. TrustTrack and Langfuse were not executed. The cosign run uploaded to the public Rekor log and verified online.

System	Status	Wrap ms	Bytes	Offline verify	Byte edit detected	Key assumption	PII in artifact	Signed output binding
Lattice receipt	measured	5.923	3210	yes	yes	other key rejected	no	yes
JSON log	baseline	0.033	1045	no	no	—	yes	no
Hash chain	baseline	0.082	1531	yes	yes	rebuild accepted	yes	no
OTel GenAI spans	measured	0.228	1366	no	no	—	yes	no
Agent Flight Recorder	measured	0.797	2328	yes	yes	mock anchor only	yes	no
AFR disclosure	measured	1.907	3357	yes	yes	mock anchor only	no	no
NovaFabric HMAC	measured	0.077	1432	yes	yes	shared secret re-signs	yes	no
NovaFabric capsule	measured	1373.053	21288	yes	yes	self-asserted key	yes	yes
cosign sign-blob	measured	4916.329	1319	online	yes	local key, Rekor upload	yes	yes
in-toto link	measured	34.149	888	yes	yes	functionary key	no	yes
Sello receipt	measured	9.336	1359	yes	yes	mock log	no	yes
Janus evidence log	measured	5	1723	yes	yes	writer key	yes	yes
TrustTrack	paper only	—	—	—	—	—	—	—
Langfuse	not run	—	—	—	—	—	—	—

Table 6: Short-prompt latency and the RAG, output-binding, ticket, and crew checks. Signed ms is the median of Lattice processing plus the provider time of the attempt that returned. Lattice ms is the median signed processing time, excluding provider time, retries, and backoff. Drift cells are unique successes / successes / attempts. Output binding counts receipts that verified and whose signed output hash matched the kept answer and changed after a one-character edit, among calls that returned a receipt. Receipt columns are the single-agent return and the crew return.

Model	Provider	Plain ms	Signed ms	Lattice ms	Drift 0	Drift 0.7	Cited	Output binding	Ticket leak	Agent rcpt	Crew rcpt
llama3.2:3b local	Ollama	278	559	2.45	1/3/3	1/3/3	3/3, 3/3	6/6	no	0	4
`minimax/minimax-m3.1-flash:free`	Minimax	3677	1489	1.69	3/10/10	1/10/10	10/10, 10/10	20/20	no	0	4
`xingchen-agi/xing4.0-29b-a4b:free`	Phala	3124	4812	1.21	4/8/10	3/3/10	8/8, 3/3	11/11	no	0	3
`nvidia/nemotron-3.5-super-v1:free`	Nvidia	675	845	1.20	1/10/10	2/10/10	10/10, 10/10	20/20	no	0	4
`nvidia/nemotron-3.5-lightning:free`	Nvidia	1319	2701	1.10	4/10/10	6/10/10	10/10, 10/10	20/20	no	0	3
`dots-studio/dots-3-note-preview:free`	AtlasCloud	1145	1275	1.50	2/10/10	3/10/10	10/10, 10/10	20/20	no	0	4
`liquid/lfm-2.5-2.6b:free`	Liquid	4179	4649	1.65	3/10/10	2/10/10	10/10, 10/10	20/20	no	fail	fail
`thinkingmachines/inkling-small:free`	—	—	—	—	—	—	—, —	—	—	—	—
`poolside/laguna-s-2.1:free`	Poolside	715	1413	1.56	1/10/10	4/10/10	10/10, 10/10	20/20	no	0	4

OOM-killed on long inputs, so each batch of 250 documents ran in its own process with a fresh Vault. Raw SSN labels are mapped when the NER map returns them unchanged. Presidio (Microsoft, 2018) used its pattern and spaCy (Honnibal et al., 2020) recognizers for its four in-scope entities, with `en_core_web_sm` rather than its default `en_core_web_lg`. Lattice is an in-process JavaScript regex and the other detectors are Python pipelines, so Figure 9 compares runtimes as well as detectors. The Lattice rows were measured with the detectors at commit `a8cc091`; Section 6.7 describes how the current detectors differ.

guardrails-detect-pii is from docs, not measured: ImportError: cannot import name 'DetectPII' from 'guardrails.hub' nemo-guardrails is from docs, not measured: nemoguardrails is not installed, and en_core_web_lg is not downloaded. The built-in sensitive-data rails call Presidio, which is scored on its own. The library also documents a separate GLiNER rail.

On ASQA dev (Stelmakh et al., 2022) (n=948), a case-sensitive Wikipedia title substring matches 525 long answers and a case-insensitive substring matches 605. Square-bracket citations appear in 2 an-

Table 7: Pure Lattice processing on the signed short prompt. The percentage is Lattice ms over Lattice ms plus the provider time of the attempt that returned; retried attempts, backoff, and pacing are excluded. Extra tokens are Lattice prompt tokens minus the direct prompt tokens. Provider delta is the signed provider time minus the direct provider time, in milliseconds, and is the latency of the longer packed prompt.

Model	ms p50	ms p95	% p50	% p95	Extra tokens	Provider delta
llama3.2:3b local	2.45	14.96	0.439	2.469	—	—
`minimax/minimax-m3.1-flash:free`	1.69	3.66	0.218	0.327	63	-2398
`xingchen-agi/xing4.0-29b-a4b:free`	1.21	3.50	0.032	0.033	83	3352
`nvidia/nemotron-3.5-super-vl:free`	1.20	2.03	0.138	0.260	80	169
`nvidia/nemotron-3.5-lightning:free`	1.10	1.96	0.053	0.082	82	1703
`dots-studio/dots-3-note-preview:free`	1.50	1.90	0.123	0.141	74	59
`liquid/lfm-2.5-2.6b:free`	1.65	4.09	0.035	0.076	61	1005
`thinkingmachines/inkling-small:free`	—	—	—	—	—	—
`poolside/laguna-s-2.1:free`	1.56	4.01	0.142	0.266	76	318

Table 8: ai4privacy English validation (Ai4Privacy, 2024), document level. Latency is the median milliseconds per document. Lattice rows use the detectors at commit `a8cc091`.

Tool	Email			US SSN			US phone			Card			ms
	P	R	F1	P	R	F1	P	R	F1	P	R	F1	
Lattice	0.955	0.979	0.967	0.325	0.436	0.372	0.316	0.333	0.324	0.000	—	—	0.007
presidio	0.974	0.999	0.986	0.334	0.502	0.401	0.295	0.504	0.372	0.000	—	—	9.413
spaCy sm	—	0.000	—	—	0.000	—	—	0.000	—	—	—	—	8.199
gliner-small	0.695	0.625	0.658	0.283	0.458	0.350	0.228	0.161	0.189	0.000	—	—	52.862
llm-guard-anonymize	0.965	0.999	0.982	0.991	0.450	0.619	0.650	0.962	0.776	0.000	—	—	142.115

swers and overlap a gold title in 0. `mustCite` accepts a gold title in 7. These three checks are different predicates. The `mustCite` count was measured at commit `a8cc091`, where an inline `[name]` or `(name)` marker containing the title counted as a citation by default. The current default never reads the answer text, so it accepts a gold title in none of these answers unless `inline: true` is passed. A local cross-encoder, `typeform/distilbert-base-uncased-mnli`, scored 696 of 948 pairs and labeled 296 entailed (empty 252, NEUTRAL 211, CONTRADICTION 189, ENTAILMENT 296). Median classification time was 820.0 ms. The premise is the gold knowledge text truncated to 1,800 characters and the hypothesis is the long answer. Pairs with an empty premise or answer were not scored. This is a support label, not a citation-string check.

LiteLLM (BerriAI, 2023) here is an in-process client, not a proxy process. Its router sent eight short prompts to two identical Ollama deployments and returned 1 distinct route across 8 calls. That is a weak determinism check. The other wrappers are a LangChain callback handler (LangChain Inc., 2023), OpenLLMetry's in-process spans (Traceloop, 2023), and a local Portkey gateway (Portkey AI, 2024). Helicone and Langfuse were not started. The latency rows are small samples (n=3–5 per arm). Each wrapper ran its calls in its own block, not interleaved with the direct arm, against `llama3.2:3b` on a CPU-only host. The min–max ranges overlap, so the p50 order reflects generation noise, not wrapper overhead.

Table 9: Gretel English finance test (Gretel.ai, 2024), same document-level rule. Gold credit-card spans in this file use the label `credit_card_number`. Lattice rows use the detectors at commit `a8cc091`.

Tool	Email			US SSN			US phone			Card			ms
	P	R	F1	P	R	F1	P	R	F1	P	R	F1	
Lattice	0.522	0.934	0.670	0.161	0.771	0.266	0.521	0.702	0.598	0.881	0.500	0.638	0.020
presidio	0.526	0.934	0.673	0.244	0.729	0.366	0.458	0.786	0.579	0.833	0.405	0.545	25.627
spaCy sm	—	0.000	—	—	0.000	—	—	0.000	—	—	0.000	—	21.623
gliner-small	0.237	0.511	0.324	0.090	0.557	0.155	0.338	0.234	0.277	0.165	0.500	0.248	156.490
llm-guard-anonymize	0.528	0.952	0.679	0.984	0.857	0.916	0.757	0.952	0.843	0.361	0.581	0.446	426.513

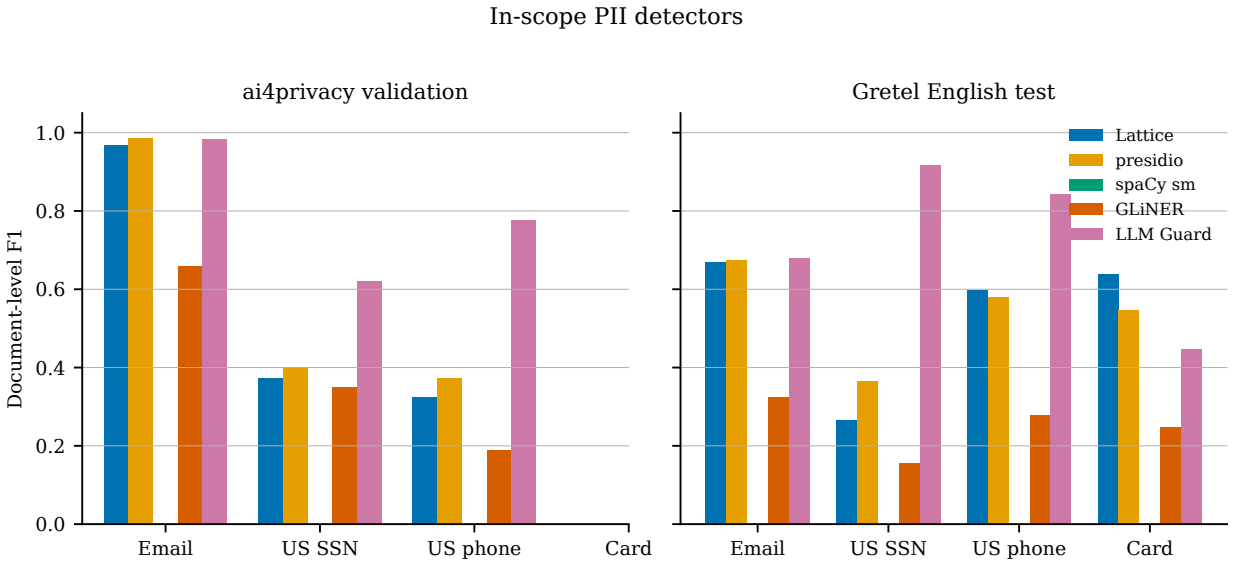


Figure 8: Document-level F1. A missing bar is an undefined F1.

6.9 Planned naturalistic evaluation with developers and auditors

The artificial evaluation establishes technical efficacy but not utility for the people who must act on the evidence. Following FEDS (Venable et al., 2016), we plan a naturalistic episode. [TODO: Lakshman and Prof. Neouchi: approve, adapt, or replace the following design, and obtain UT Dallas IRB determination before recruiting.]

- *Participants:* 12–20 participants in two roles, software developers who build LLM features and auditors or compliance analysts.
- *Tasks:* (a) developers instrument an existing LLM feature with receipts, measuring time to adoption and code changes (DR5); (b) auditors receive a set of runs, some tampered and some with policy violations, as either conventional logs or receipts with the command-line tool, and must identify which runs are trustworthy and which violated policy (DR1, DR3).
- *Design:* within-subject comparison of logs and receipts with counterbalanced order.
- *Measures:* task accuracy and time, perceived usefulness and ease of use, and semi-structured interview

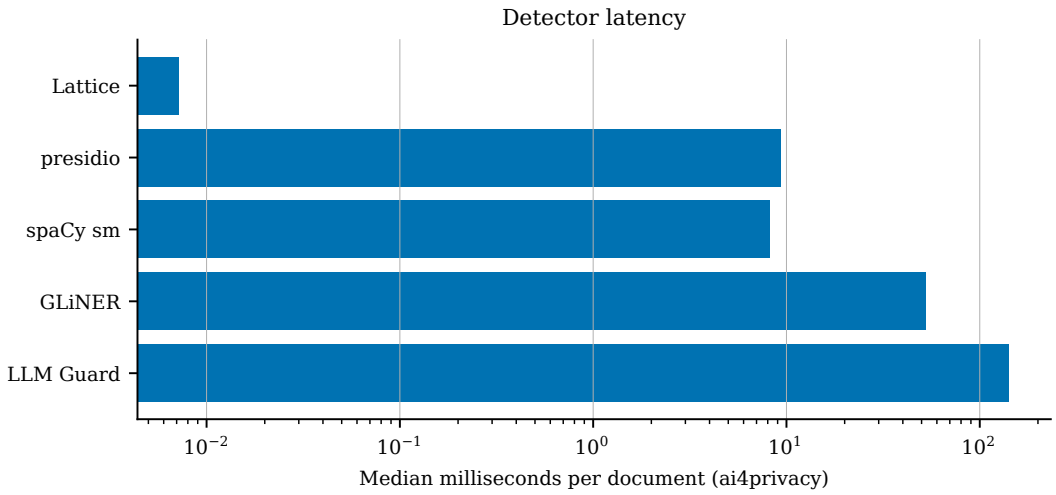


Figure 9: Median detector time per ai4privacy document. The axis is logarithmic. Lattice runs in-process in JavaScript; the other detectors are Python pipelines.

Table 10: Local wrappers around Ollama. Marker detection scans the captured log or span. Portkey's log route failed, so that row does not show what a stored log would contain. None of these logs is signed.

System	Status	n	p50 ms	min–max ms	Bytes	Markers in capture	Offline verify	Signed output binding
ollama-direct	measured	5	1060.2	435–1833	—	—	—	—
litellm	measured	5	3964.8	587–6363	2408	prompt stored	no	no
langchain-callback	measured	5	1936.3	844–8321	6893	yes	no	no
openllmetry	measured	3	1253.1	1211–1329	3346	yes	no	no
portkey-gateway	measured	5	6281.0	1099–8688	1953	log unread	no	no
helicone	from docs, not measured	—	—	—	—	—	—	—
langfuse	from docs, not measured	—	—	—	—	—	—	—

data on fit with audit procedures and frameworks such as ISO/IEC 42001 (ISO/IEC, 2023).

- *Analysis:* paired tests for accuracy and time; thematic analysis of interviews to refine the design principles.

7 Discussion

7.1 Contribution to IS knowledge

In terms of the knowledge contribution framework of Gregor and Hevner (2013), this study offers an improvement: a new solution to a known problem, auditable record keeping, for a new application context. Its contribution has two levels. At the level of nascent design theory (Gregor and Jones, 2007), it contributes five requirements and four principles for run-level AI accountability that apply beyond the specific artifact. At the level of the instantiation, it contributes an open-source artifact and a reusable evaluation harness, together with quantitative evidence about which principles the instantiation realizes. The evaluation also yields design knowledge that is easy to overlook. A self-attested receipt guarantees integrity of what was signed, not freshness, unless the issuer set an expiry or the verifier asks for an audience, a nonce, or a maximum age.

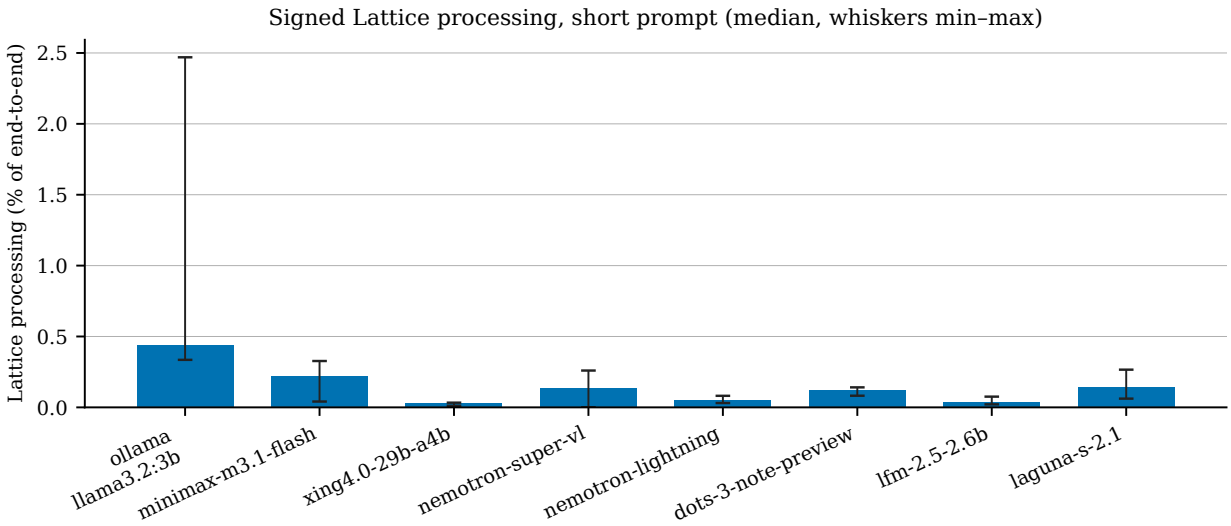


Figure 10: Signed Lattice processing time as a percentage of end-to-end time, where end-to-end time is Lattice processing plus the provider time of the attempt that returned. Retried attempts, backoff, and pacing are excluded. The bar is the median and the whiskers run from the minimum to the maximum. The local bar is wall time minus the localhost HTTP call in the Ollama run of Section 6.7.

A receipt attests only to what is hashed into it. Lattice receipts omit the task text and the policy, so they show what came back and from which route, not what the system was asked. A hash of an input does not show that the provider received the input, as the dropped images on OpenAI-shaped adapters show, and a hash hides a value only when the value is hard to guess. Linking receipts does not by itself reveal a missing tail. A closing receipt that lists members does, but only for verifiers that enforce it. A redaction manifest is a claim unless the signed bytes are the masked form, which the leakage scan checks. Offline replay of stored outputs is an output binding, not a re-execution, and other signed record systems offer the same binding (Figure 15). These lessons transfer to any organization that designs audit trails for AI-enabled processes.

The study also connects IS research on AI governance (Berente et al., 2021; Mäntymäki et al., 2022; Papagiannidis et al., 2025) with the security literature on tamper-evident records (Schneier and Kelsey, 1999; Crosby and Wallach, 2009; Laurie et al., 2013). Governance research has described *what* organizations must be able to show. This study demonstrates *how* an information system can be designed so that the evidence is produced as a by-product of normal operation rather than as a separate compliance activity, extending the idea of sociotechnical envelopment (Asatiani et al., 2021) to the evidentiary layer.

7.2 Implications for practice

For managers and compliance officers, the findings suggest that run-level evidence can be produced at negligible runtime cost, which removes a common argument against comprehensive logging of AI use. Evidence of this kind maps onto record-keeping duties in the EU AI Act (European Parliament and Council of the European Union, 2024), onto the measurement and management functions of the NIST AI RMF (Tabassi, 2023), and onto the documented controls an ISO/IEC 42001 management system expects (ISO/IEC, 2023).

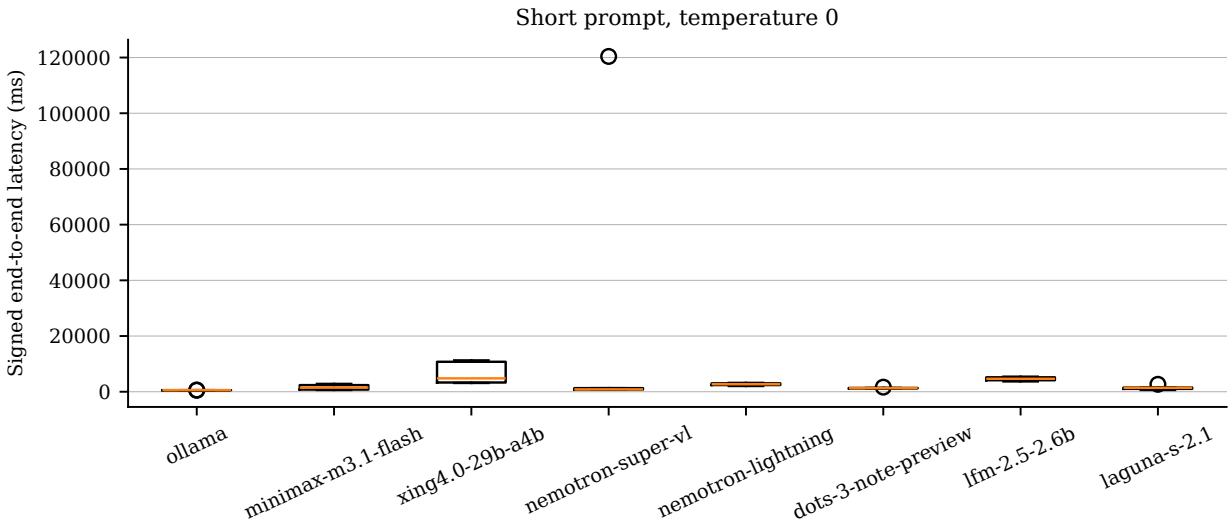


Figure 11: Signed short-prompt latency. Ollama is five trials. Each hosted row is the successful signed trials, as Lattice processing plus the provider time of the attempt that returned.

It can supply concrete artifacts for the application layer of LLM audits (Mökander et al., 2023). Three cautions follow from the evaluation. First, an issuer-signed receipt proves integrity, not honesty. Organizations that need protection against a dishonest operator should combine receipts with receiver-side attestations or transparency logs, as in Sello (Figuera, 2026) or Certificate Transparency (Laurie et al., 2013). Second, auditors should insist on chain verification and on the catalog version used for routing, and should keep the task text and the policy alongside the receipt, because the receipt does not commit to them. Third, pattern-based personal-data checks are not a complete privacy control. On the synthetic set they behave as specified, but on public datasets their SSN and phone precision and recall were low (Tables 8 and 9), because other identifiers share the same digit shapes. They also miss types they do not claim, numbers written after an identifier label, and bare numbers without a type cue.

For developers, the design principles suggest a pattern: configure signing once, commit to hashes rather than content, make consequential decisions pure functions that can be recomputed, and verify before loading.

7.3 Limitations and future research

The evaluation has several limitations. Some of them were first reported in an evaluation filed as a public issue (Trace, 2026).

- *Mocked and live providers.* The controlled tables use a simulated provider. Sections 6.7 and 6.8 run a local model and free hosted models. The hosted sample is a free tier: it rate-limits, it does not represent paid latency, and one catalogued model rejects ordinary chat calls. The OpenAI-compatible adapter used for the live runs sends no temperature or token cap, so a fetch wrapper added both. Provider-time differences between a packed prompt and a one-line prompt are too noisy to attribute to the extra tokens.

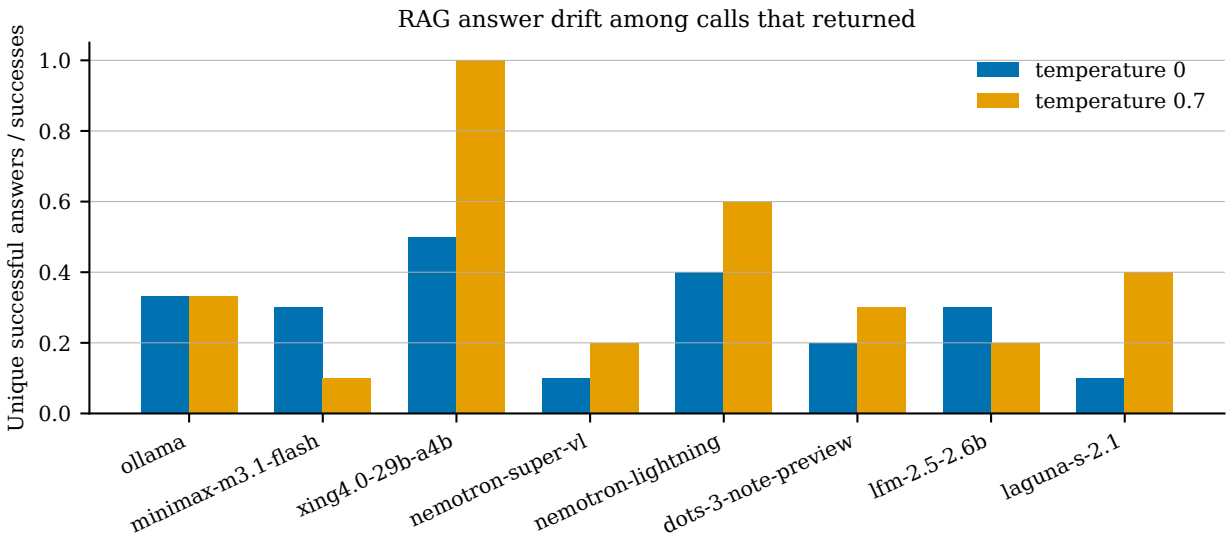


Figure 12: Distinct successful RAG answers divided by the number of successful calls. Ollama used n=3. Hosted rows use the successes recorded for that model.

- *The task, prompt, and policy are not signed.* A receipt commits to input-artifact hashes, the output hash, the contract hash, and the selected route. Two runs that differ only in their task text, and that return the same output with the same token usage, yield receipts that differ only in identifiers and time. The receipt does not record why a route was chosen.

- *The single-agent loop returns no receipt.* `ai.run` and `runAgentCrew` return receipts. `ai.runAgent` does not; with a signer on its intent, its checkpoint receipts reach only a configured tracer, and without a tracer they are discarded.

- *Artifact transport on OpenAI-shaped adapters.* The OpenAI, OpenAI-compatible, xAI, OpenRouter, and LM Studio adapters send artifacts as text parts holding JSON and include an artifact's value only when it is a string. A data-URL string is inlined as text, other values such as binary image bytes and JSON objects are dropped from the request, and the receipt still hashes the dropped input. In the vision probe of Section 6.8, the Lattice arm sent the image as inlined text and misread a solid red image that a direct image request read correctly. Only the Anthropic and Gemini adapters send native image parts.

- *Unsalted hashes.* Input and output hashes are unsalted SHA-256, so a holder of a receipt can confirm a guessed low-entropy value, such as an SSN or a yes/no decision.

- *Freshness binding is opt-in.* Audience, nonce, and a raised version floor are not required for verification. A receipt with no `expiresAt` does not expire unless the caller passes `maxAgeMs`. There is no replay cache. Advisories report unchecked bindings as warnings or information. They do not reject.

- *Open chains can still be tail-truncated.* Without `expectedCount` or a closing member list, truncation produces a warning, not a failure. Crew chains carry a closing member list, but verifiers in the 1.6.x line do not enforce it.

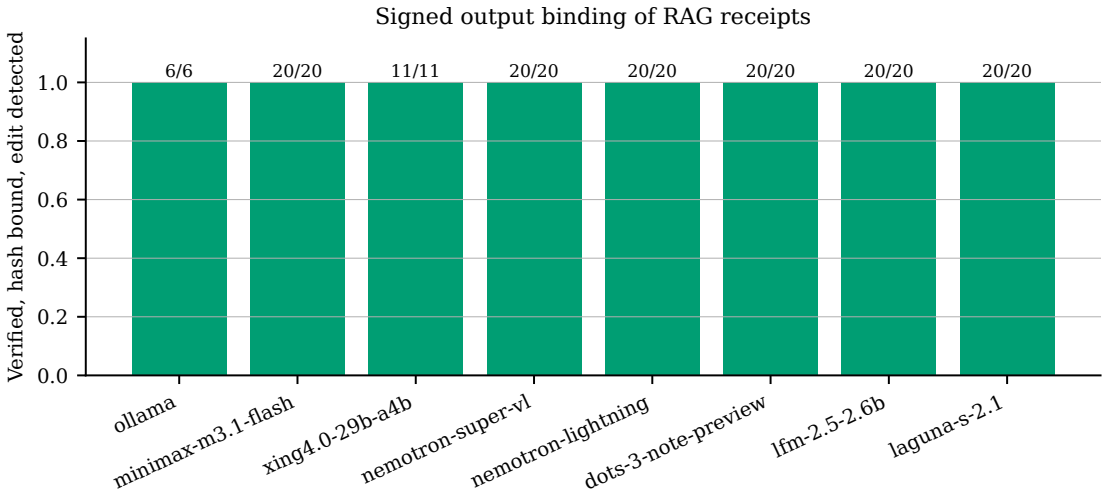


Figure 13: Signed output binding on receipts from calls that returned. A receipt counts when it verified, its signed output hash matched the kept answer, and a one-character edit of that answer changed the hash. `replayOffline` returns the outputs the caller supplies, so this is a hash-binding check, not a re-run of the model. Rate-limited calls with no receipt are left out of the denominator.

- *Signer failure is visible, not fail-closed by default.* If the signer throws, the run still returns its result, which carries `receiptError`, unless `onSignerError` is set to throw. A crew-start signing failure always throws, because no agent work has run. A run with no signer emits no receipt.
- *Regex personal-data checks.* Detectors cover email, United States SSNs, payment cards, and United States phone numbers. Names, addresses, and non-United States formats are out of scope. An identifier label hides a real number written after it, bare nine- and ten-digit numbers need a type cue, and lists of long numeric identifiers can trip the card check. The public-data rows were measured with the rules at commit `a8cc091`, not with the rules described in Section 6.4.
- *must-cite is syntactic.* By default it reads only structured citation lists. Prose checks are opt-in. It does not decide whether a cited artifact supports the prose.
- *Self-attestation and synthetic data.* Receipts are signed by the issuing runtime. The policy evaluation used synthetic data, and its labeled cases were chosen with the rules they test in view.
- *No human evaluation yet* (Sect. 6.9).

Future research can commit the task, the packed prompt, and the policy in the receipt, replace unsalted hashes with salted or keyed commitments, send images to OpenAI-shaped providers as native parts or record when an artifact could not be sent, combine issuer receipts with receiver attestations and transparency logs, study how auditors use receipts in practice, and examine how receipts can serve as evidence in conformity assessments under emerging AI regulation.

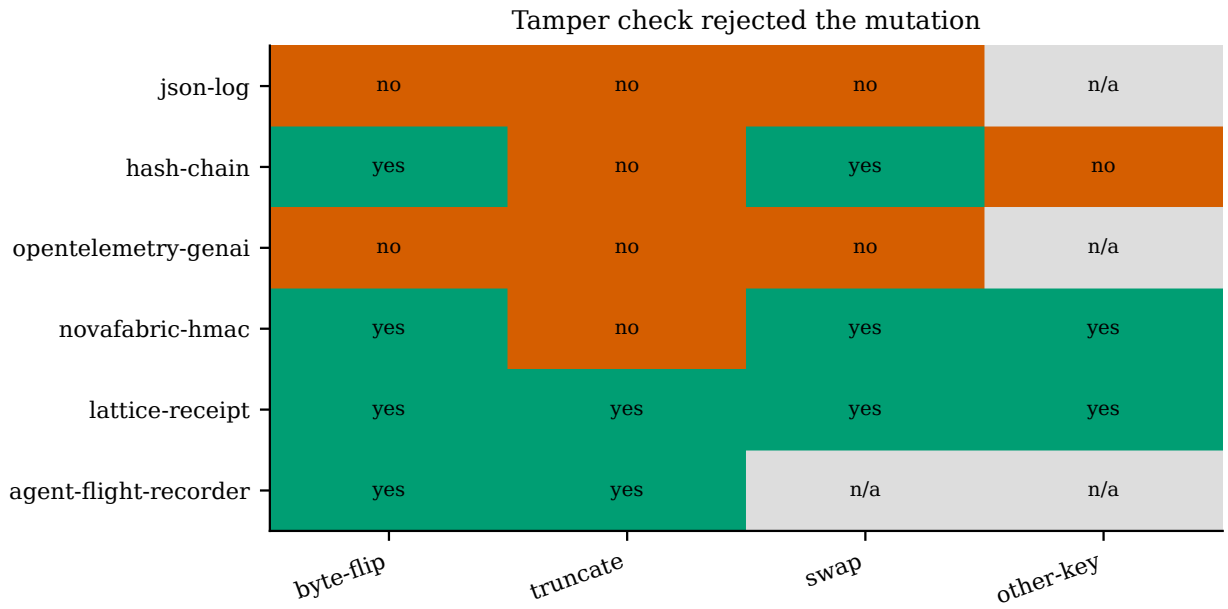


Figure 14: Whether the system's own check rejects the mutation. Gray means that class was not applied. Truncating a hash chain leaves a valid prefix. A byte flip inside a JSON string, a dropped tail record, and a swapped response all remain valid JSON, so a parser does not catch them.

8 Conclusion

AI-enabled information systems need records that auditors can verify and operators can reproduce. Following design science research, this study derived requirements and principles for run-level AI accountability, instantiated them in Lattice, and evaluated the instantiation offline. Where the harness required rejection, single-receipt tamper attempts were rejected, recorded outputs matched the signed output hash, routing policies were not violated, and signing added milliseconds per run (Table 4). Freshness binding and open-chain length remain opt-in, and crew chains carry a closing member list. Personal data matched by the privacy check is masked before signing. The receipts do not commit to the task text or the policy, the single-agent loop returns none, and OpenAI-shaped adapters can hash an image the provider never received. Together, the measured results, the remaining opt-ins, and these gaps are design knowledge for organizations that want AI evidence produced during the run.

Acknowledgements

The personal-data evaluation uses the pii-masking-300k dataset by Ai4Privacy (Ai4Privacy, 2024) under its terms for academic, non-commercial use, which require acknowledgement, and the Gretel synthetic finance dataset (Gretel.ai, 2024) under the Apache-2.0 license. The citation evaluation uses the ASQA dataset (Stelmakh et al., 2022) under the Apache-2.0 license. Only aggregate scores from these datasets are published with the artifact.

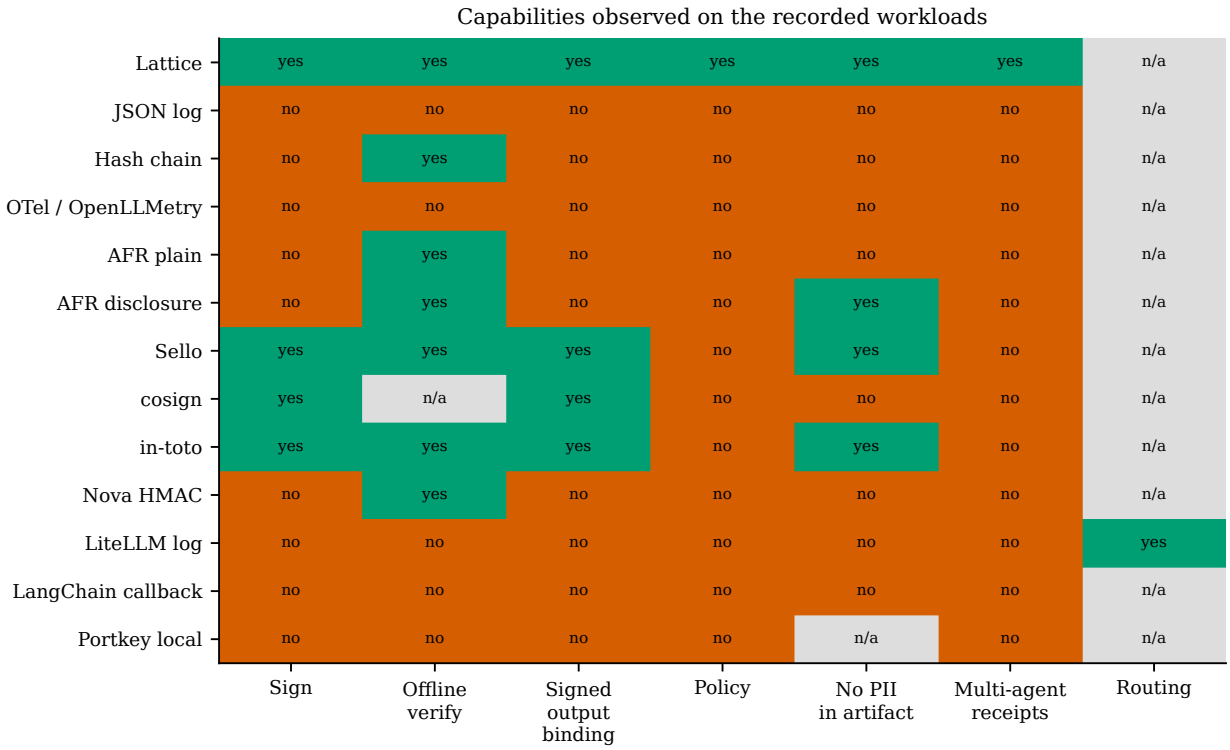


Figure 15: Capabilities observed on these workloads. Gray is not measured here. Signed output binding is yes when the system signs the record and its own verifier rejected a byte edit of the stored record; for Lattice the signed output hash is checked by re-hashing the kept answer. cosign's offline-verify cell is gray because the measured `verify-blob` checked the Rekor log online. LiteLLM routing is eight calls across two identical local deployments, which returned one route. Lattice routing is gray because each live call used one configured provider. Portkey's stored-log endpoint returned HTTP 500, so the PII cell is gray. LiteLLM's callback log stored the prompt text. The single-agent Lattice return carried no receipt; the crew return did.

Declarations

Funding No funding was received for conducting this study.

Competing interests Lakshman Turlapati is the author and maintainer of Lattice and is affiliated with Full Self Browsing, which publishes Lattice. Rabih Neouchi declares no competing interests.

Data and code availability The artifact and the evaluation harness, including the result files the tables are generated from, are available at `https://github.com/fullselfbrowsing/Lattice`. [TODO: Push the evaluation branch and archive a tagged release with a DOI (e.g., Zenodo) before submission.]

Author contributions [TODO: CRediT statement, e.g., Conceptualization: L.T., R.N.; Software: L.T.; Methodology: L.T., R.N.; Writing: L.T.; Supervision: R.N.]

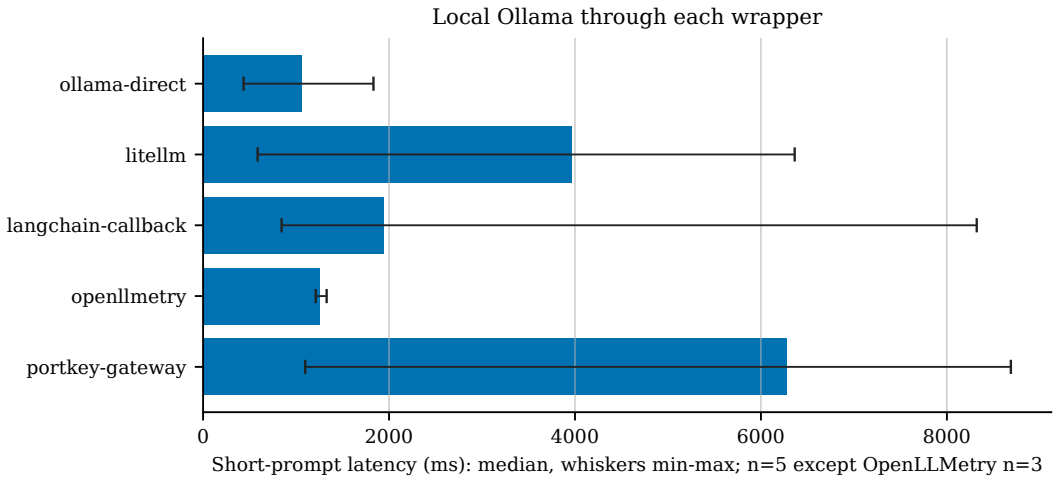


Figure 16: Short-prompt latency through each local wrapper: median, with whiskers from the minimum to the maximum. OpenLLMetry used n=3. The other measured rows used n=5. The arms ran one after another on a CPU-only host, so the order is not a ranking of wrapper overhead.

Ethics approval Not applicable to the reported evaluation, which involved no human participants. [TODO: Required for the planned study in Sect. 6.9.]

A Reproducibility

The narrative quotes the harness through macros generated by `eval/scripts/06-report.py`. The latency and size results (Tables 11, 12, and 13; Figures 3 and 4) come from the original measurement run on Node v24.22.1, Intel(R) Xeon(R) Processor (4 cores, 15.6 GB), platform linux 6.12.94+, Lattice 1.6.0, commit `45493340b1c26c0e30307c558bea6a0bd17f7ec8`, at 2026-10-21T16:29:36.012Z. The functional experiments (tamper, replay, policy, and verifier checks) do not depend on timing. They ran against the library source at commit `6d15f0a95acbe524eed02b68bc63c9219c536462` on Node v24.14.1, Apple M4 Max (16 cores, 64 GB), platform darwin 25.6.0, at 2026-10-22T10:12:04.327Z, and the test counts below come from the same environment. The library tests passed 1596 of 1596 (0 failed). The command-line tests passed 190 of 190 (0 failed). Those counts document that the suite ran. They are not the argument for the design.

Provider calls were mocked. No API credential was read from the environment. Regenerate with `eval/run-all.sh` and `pnpm bench:receipts`. Both re-measure latency on the host that runs them. The repository benchmark, which is separate from the harness tamper suite, writes `paper/bench/results.json`. The live and comparison results of Sections 6.7 and 6.8 are generated from `eval/realworld/results` by `eval/realworld/scripts/05-charts.py` and `12-charts.py`.

Table 11: Per-run latency (ms) with a mocked OpenAI-shaped provider, 1000 trials after 100 warm-up. Overhead = signed run − unsigned run. Latency environment (`env-latency.json`): Node v24.22.1, Intel(R) Xeon(R) Processor x4, linux 6.12.94+, commit 4549334.

Config	plain p50	run p50	signed p50	signed p95	signed p99	verify p50
text-1KB	0.056	0.075	0.263	0.590	0.826	0.180
text-10KB	0.074	0.092	0.271	0.591	0.804	0.157
text-100KB	0.308	0.320	0.672	1.238	1.545	0.165
text-1000KB	3.687	3.589	5.440	7.665	8.823	0.226
image-100KB	0.064	0.088	0.445	0.877	4.850	0.182
image-1000KB	0.078	0.087	1.534	5.076	6.479	0.199
multimodal-5art	0.092	0.118	1.356	3.508	7.700	0.187
text-x1	0.044	0.065	0.233	0.456	0.827	0.153
text-x10	0.172	0.215	1.266	1.746	2.553	0.197
text-x50	0.552	0.758	6.020	7.387	10.009	0.225

Isolated: createReceipt p50 0.059 / p99 0.223; verifyReceipt p50 0.125 / p99 0.232; receiptCid p50 0.0508

Table 12: Receipt envelope size vs. input payload. Latency environment (`env-latency.json`): Node v24.22.1, Intel(R) Xeon(R) Processor x4, linux 6.12.94+, commit 4549334.

Config	payload B	artifacts	receipt B
text-1KB	1024	1	1183
text-10KB	10240	1	1183
text-100KB	102400	1	1183
text-1000KB	1024000	1	1183
image-100KB	102400	2	1271
image-1000KB	1024000	2	1271
multimodal-5art	618496	5	1539
text-x1	1024	1	1183
text-x10	10240	10	1987
text-x50	51200	50	5559

Table 13: Chained receipts (parentReceiptCid): size and verification cost by depth (30 trials). Latency environment (`env-latency.json`): Node v24.22.1, Intel(R) Xeon(R) Processor x4, linux 6.12.94+, commit 4549334.

Depth	total B	B/link	build p50 ms	verify-all p50 ms
1	1023	1023	0.202	0.303
2	2170	1085	0.268	0.449
4	4464	1116	0.526	0.888
8	9052	1132	1.076	1.688
16	18228	1139	1.996	3.312
32	36580	1143	3.889	6.266
64	73284	1145	7.595	12.599
128	146692	1146	14.913	25.074
256	293508	1147	30.116	49.059

Table 14: Offline replay over recorded fixtures, per adapter. Fixtures are verified with the public keyset written next to them. Functional environment (`env-functional.json`): Node v24.14.1, Apple M4 Max x16, darwin 25.6.0, commit 6d15f0a.

Adapter	runs	verified	hash match	live rerun match	altered detected
openai	50	50	50	50	50
openai-compatible	50	50	50	50	50
anthropic	50	50	50	50	50
gemini	50	50	50	50	50
xai	50	50	50	50	50
openrouter	50	50	50	50	50
lm-studio	50	50	50	50	50

Table 15: Policy checks on a synthetic labeled set (100 samples per label). The original set was written alongside the detectors. The added set holds PII in support and CRM prose, tables, and CSV rows, more card networks, and uncued numbers that are not PII. Rows with only the original negatives leave the added negatives out. The mustCite cases are the original set, default-behavior cases, and opt-in prose checks. The prose-by-default row scores the original mustCite cases against labels that expect prose to be read by default; inline-text-only and hallucinated-cite-string differ, because the default reads prose only on opt-in. The set is a regression suite, not a prevalence-weighted estimate. Functional environment (`env-functional.json`): Node v24.14.1, Apple M4 Max x16, darwin 25.6.0, commit 6d15f0a.

Set	n	precision	recall	FP	FN
noPII, in-scope PII types, all negatives	5500	1	0.9429	0	200
noPII, in-scope PII types, original negatives only	4500	1	0.9429	0	200
noPII, all PII types, all negatives	6100	1	0.8083	0	786
noPII, all PII types, original negatives only	5100	1	0.8083	0	786
noPII, out-of-scope types, all negatives	2600	1	0.0233	0	586
noPII, original labels only (in-scope)	2400	1	1	0	0
noPII, added labels only (in-scope)	3100	1	0.9048	0	200
added uncued numeric negatives	1000	0 flagged			
mustCite agreement, all cases	36	36/36 agree with label			
mustCite, default behavior	22	22/22			
mustCite, opt-in prose checks	14	14/14			
mustCite, original cases	15	15/15			
mustCite, original cases, prose-by-default labels	15	13/15 (2 labels differ)			
routing determinism	20000	identical 1.0000, permuted same 1.000, violations 0			

Table 16: Tamper suite over 70 receipts from mocked `ai.run` calls across the seven adapters: rejections per mutation and the error kind `verifyReceipt` returned. Rows in the control-accept and replay classes are expected to verify (there is no replay cache). Last block: 50 linked chains of 8 receipts, rejection rate by per-receipt `verifyReceipt` / by the library's `verifyChain` with no `expectedCount` (except the expectedCount row). Accepted chains list the advisory codes `verifyChain` returned. Functional environment (`env-functional.json`): Node v24.14.1, Apple M4 Max x16, darwin 25.6.0, commit 6d15f0a.

Class	Mutation	Rejected	Kind
body-edit-keep-sig	edit-outputHash	70/70	signature-invalid:70
body-edit-jcs-keep-sig	edit-outputHash	70/70	signature-invalid:70
body-edit-attacker-sig-legit-kid	edit-outputHash	70/70	signature-invalid:70
body-edit-keep-sig	edit-inputHashes	70/70	signature-invalid:70
body-edit-jcs-keep-sig	edit-inputHashes	70/70	signature-invalid:70
body-edit-attacker-sig-legit-kid	edit-inputHashes	70/70	signature-invalid:70
body-edit-keep-sig	edit-route-provider	70/70	signature-invalid:70
body-edit-jcs-keep-sig	edit-route-provider	70/70	signature-invalid:70
body-edit-attacker-sig-legit-kid	edit-route-provider	70/70	signature-invalid:70
body-edit-keep-sig	edit-model	70/70	signature-invalid:70
body-edit-jcs-keep-sig	edit-model	70/70	signature-invalid:70
body-edit-attacker-sig-legit-kid	edit-model	70/70	signature-invalid:70
body-edit-keep-sig	edit-usage-cost	70/70	signature-invalid:70
body-edit-jcs-keep-sig	edit-usage-cost	70/70	signature-invalid:70
body-edit-attacker-sig-legit-kid	edit-usage-cost	70/70	signature-invalid:70
body-edit-keep-sig	edit-verdict	70/70	signature-invalid:70
body-edit-jcs-keep-sig	edit-verdict	70/70	signature-invalid:70
body-edit-attacker-sig-legit-kid	edit-verdict	70/70	signature-invalid:70
body-edit-keep-sig	edit-issuedAt	70/70	signature-invalid:70
body-edit-jcs-keep-sig	edit-issuedAt	70/70	signature-invalid:70
body-edit-attacker-sig-legit-kid	edit-issuedAt	70/70	signature-invalid:70
body-edit-keep-sig	edit-runId	70/70	signature-invalid:70
body-edit-jcs-keep-sig	edit-runId	70/70	signature-invalid:70
body-edit-attacker-sig-legit-kid	edit-runId	70/70	signature-invalid:70
body-edit-keep-sig	drop-redactions	70/70	version-mismatch:70
body-edit-jcs-keep-sig	drop-redactions	70/70	version-mismatch:70
body-edit-attacker-sig-legit-kid	drop-redactions	70/70	version-mismatch:70
body-edit-keep-sig	edit-redactionPolicy	70/70	signature-invalid:70
body-edit-jcs-keep-sig	edit-redactionPolicy	70/70	signature-invalid:70
body-edit-attacker-sig-legit-kid	edit-redactionPolicy	70/70	signature-invalid:70
body-edit-keep-sig	add-field	70/70	canonicalization-mismatch:70
body-edit-jcs-keep-sig	add-field	70/70	signature-invalid:70
body-edit-attacker-sig-legit-kid	add-field	70/70	signature-invalid:70
body-edit-keep-sig	unicode-nfd	70/70	signature-invalid:70
body-edit-jcs-keep-sig	unicode-nfd	70/70	signature-invalid:70
body-edit-attacker-sig-legit-kid	unicode-nfd	70/70	signature-invalid:70
encoding	key-reorder	70/70	canonicalization-mismatch:70
encoding	whitespace	70/70	canonicalization-mismatch:70
signature	sig-bitflip	70/70	signature-invalid:70

Table 16, continued

Class	Mutation	Rejected	Kind
signature	sig-swap-same-key	70/70	signature-invalid:70
signature	sig-truncate	70/70	signature-invalid:70
signature	sig-empty-list	70/70	envelope-malformed:70
signature	payload-truncate	70/70	envelope-malformed:70
signature	payloadType-change	70/70	envelope-malformed:70
key	keyid-to-other-trusted	70/70	signature-invalid:70
key	keyid-unknown	70/70	key-not-found:70
key	forge-attacker-key-own-kid	70/70	key-not-found:70
key	resign-revoked-key	70/70	key-revoked:70
key	resign-other-key-body-kid-mismatch	70/70	signature-invalid:70
downgrade	version-v1-resigned-valid-key	70/70	schema-version-too-low:70
downgrade	version-absent-resigned-valid-key	70/70	schema-version-too-low:70
control-accept	untouched	0/70	accepted(active):70
control-accept	retired-key-legit-signature	0/70	accepted(retired):70
control-accept	version-v1.1-resigned-valid-key (key holder)	0/70	accepted(active):70
replay	same-receipt-presented-again (no nonce/audience)	0/70	accepted(active):70
Chain attacks: rejection rate, per-receipt verify / verifyChain			
chain	swap-adjacent-links	0.00 / 0.00	accepted: tail-truncation-unchecked, audience-absent, expiry-absent, nonce-absent
chain	drop-middle-link	0.00 / 1.00	rejected: orphan-link
chain	splice-foreign-link	0.00 / 1.00	rejected: orphan-link
chain	truncate-tail	0.00 / 0.00	accepted: tail-truncation-unchecked, audience-absent, expiry-absent, nonce-absent
chain	truncate-tail-expectedCount	0.00 / 1.00	rejected: count-mismatch
chain	replace-root	0.00 / 1.00	rejected: orphan-link

Table 17: Verifier opt-ins, structured advisories, chain closure, crew receipts, and signer-failure modes: 71 of 71 checks pass. Result gives the error kind, advisory, or exception the check expected. The two base-1.6.0 rows run the released 1.6.0 verifier on a chain minted by the library at the functional commit. Functional environment (`env-functional.json`): Node v24.14.1, Apple M4 Max x16, darwin 25.6.0, commit 6d15f0a.

Check	Result
plain-verifies	pass
advisories-on-unbound-receipt	pass: 3 info advisories
advisory-shape-structured	pass: {code, severity, message}
current-version-no-version-advisory	pass
bound-accepts-matching-options	pass
audience-mismatch	pass: audience-mismatch
nonce-mismatch	pass: nonce-mismatch
expired	pass: receipt-expired

Table 17, continued

Check	Result
minVersion-v1.4-accepts-current	pass
bound-unchecked-audience-warns	pass: warn:audience-unchecked
bound-unchecked-nonce-warns	pass: warn:nonce-unchecked
nonce-value-never-echoed	pass: not echoed
expiresAt-set-no-expiry-advisory	pass
bound-and-checked-no-advisories	pass: 0 advisories
maxAgeMs-silences-expiry-absent	pass
minVersion-floor-rejects-v1.1	pass: schema-version-too-low
v1.1-still-verifies-without-minVersion	pass
version-below-current-info-effective-floor	pass: info:version-below-current
version-below-current-kept-at-baseline-minVersion	pass: info:version-below-current
truncate-without-expectedCount	pass: accepted
truncate-with-expectedCount	pass: count-mismatch
expectedCount-silences-tail-advisory	pass: accepted
requireClosure-rejects-unclosed-chain	pass: count-mismatch
closed-chain-accepts	pass
closed-chain-no-tail-advisory	pass
closed-chain-dropped-leaf	pass: count-mismatch
closed-chain-dropped-closer	pass: count-mismatch
requireClosure-accepts-closed-chain	pass
partial-subset-accepts-with-closure-partial-warn	pass: warn:closure-partial
partial-full-set-no-closure-partial	pass: accepted
subset-without-partial-rejected	pass: count-mismatch
partial-requires-closer	pass: count-mismatch
partial-rejects-unlisted-member	pass: count-mismatch
createReceipt-rejects-malformed-member-cid	pass: TypeError
createReceipt-rejects-non-array-member-list	pass: TypeError
createReceipt-rejects-duplicate-member-cid	pass: TypeError
createReceipt-rejects-unsorted-member-list	pass: TypeError
createReceipt-rejects-unknown-closure-mode	pass: TypeError
createReceipt-rejects-closure-on-non-root	pass: TypeError
malformed-chainClosure-mode-verifyReceipt	pass: version-mismatch
malformed-chainMemberCids-string-verifyReceipt	pass: version-mismatch
malformed-chainMemberCids-non-string-verifyReceipt	pass: version-mismatch
malformed-root-closure-verifyChain-link-invalid	pass: link-invalid(version-mismatch)
malformed-closer-verifyChain-link-invalid	pass: link-invalid(version-mismatch)
malformed-closer-partial-link-invalid	pass: link-invalid(version-mismatch)
duplicate-member-cid-verifyChain	pass: count-mismatch
non-root-closure-receipt-verifies-alone	pass
non-root-closure-verifyChain-rejected	pass: count-mismatch
non-root-closure-partial-rejected	pass: count-mismatch
root-only-closure-unclosed-root	pass: count-mismatch
crew-receipts-n-plus-2	pass: 5 receipts for 3 agents
crew-start-declares-closure-closer-lists-members	pass: closer lists 4
crew-chain-verifies-requireClosure	pass

Table 17, continued

Check	Result
crew-closer-verdict-success	pass: success
crew-dropped-leaf-count-mismatch	pass: count-mismatch
crew-dropped-closer-count-mismatch	pass: count-mismatch
crew-partial-disclosure-warns	pass: warn:closure-partial
crew-budget-exceeded-closer-execution-failed	pass: crew-budget-exceeded / execution-failed
crew-child-mint-failure-default-receiptError	pass: signer-failed
crew-child-mint-failure-flag-receiptError	pass: signer-failed
crew-child-mint-failure-throw-ReceiptSigningError	pass: ReceiptSigningError
crew-child-mint-throw-stops-crew	pass: 2 provider calls
crew-closer-mint-failure-flagged-chain-unverifiable	pass: count-mismatch
crew-start-failure-default-ReceiptSigningError	pass: ReceiptSigningError
crew-start-failure-flag-ReceiptSigningError	pass: ReceiptSigningError
crew-start-failure-throw-ReceiptSigningError	pass: ReceiptSigningError
base-1.6.0-verifier-accepts-new-crew-chain	pass: latticeVersion 1.6.0
base-1.6.0-verifier-does-not-enforce-closure (documented)	pass: accepted
signer-flag-default	pass: signer-failed
signer-throw	pass: ReceiptSigningError
no-signer-no-receipt	pass

References

Ai4Privacy (2024). pii-masking-300k. Hugging Face dataset, revision c8c77895a005822682b66ab547fc0422579bc1d3. Academic research and non-commercial use with acknowledgement; no redistribution. `https://huggingface.co/datasets/ai4privacy/pii-masking-300k`.

Amershi, S., Begel, A., Bird, C., DeLine, R., Gall, H., Kamar, E., Nagappan, N., Nushi, B., and Zimmermann, T. (2019). Software engineering for machine learning: A case study. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 291–300. IEEE.

Arslan, M. (2026). Janus: Evidence-before-effect sagas and offline-verifiable provenance for agentic LLMs. arXiv preprint arXiv:2609.38266.

Asatiani, A., Malo, P., Nagbøl, P. R., Penttinen, E., Rinta-Kahila, T., and Salovaara, A. (2021). Sociotechnical envelopment of artificial intelligence: An approach to organizational deployment of inscrutable artificial intelligence systems. *Journal of the Association for Information Systems*, 22(2):325–352.

Baskerville, R., Baiyere, A., Gregor, S., Hevner, A., and Rossi, M. (2018). Design science research contributions: Finding a balance between artifact and theory. *Journal of the Association for Information Systems*, 19(5):358–376.

Berente, N., Gu, B., Recker, J., and Santhanam, R. (2021). Managing artificial intelligence. *MIS Quarterly*, 45(3):1433–1450.

Bernstein, D. J., Duif, N., Lange, T., Schwabe, P., and Yang, B.-Y. (2012). High-speed high-security signatures. *Journal of Cryptographic Engineering*, 2(2):77–89.

BerriAI (2023). LiteLLM: Call 100+ LLM APIs in the OpenAI format. Software, online documentation. `https://github.com/BerriAI/litellm`.

Bindschaedler, L., Botha, Q., and Siebenbrunner, C. (2026). Agent flight recorder: Tamper-evident audit trails with on-chain anchoring for long-horizon tool-using agents. In *Proc. IEEE Int. Conf. Blockchain Computing and Applications (BCCA)*. arXiv:2609.01931, `https://arxiv.org/abs/2609.01931`.

Bommasani, R., Hudson, D. A., Adeli, E., Altman, R., Arora, S., et al. (2021). On the opportunities and risks of foundation models. arXiv preprint arXiv:2108.07258.

Bray, T. (2015). The I-JSON message format. RFC 7493, RFC Editor.

Brundage, M., Avin, S., Wang, J., Belfield, H., Krueger, G., Hadfield, G., et al. (2020). Toward trustworthy AI development: Mechanisms for supporting verifiable claims. arXiv preprint arXiv:2004.07213.

Chen, L., Zaharia, M., and Zou, J. (2023). FrugalGPT: How to use large language models while reducing cost and improving performance. arXiv preprint arXiv:2305.05176.

Crosby, S. A. and Wallach, D. S. (2009). Efficient data structures for tamper-evident logging. In *18th USENIX Security Symposium (USENIX Security 09)*, pages 317–334. USENIX Association.

European Parliament and Council of the European Union (2024). Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act). Official Journal of the European Union, L series. `https://eur-lex.europa.eu/eli/reg/2024/1689/oj`.

Felzmann, H., Fosch Villaronga, E., Lutz, C., and Tamò-Larrieux, A. (2019). Transparency you can trust: Transparency requirements for artificial intelligence between legal norms and contextual concerns. *Big Data & Society*, 6(1).

Figuera, J. (2026). Notarized agents: Receiver-attested confidential receipts for AI agent actions. arXiv preprint arXiv:2606.04193.

Gebru, T., Morgenstern, J., Vecchione, B., Vaughan, J. W., Wallach, H., Daumé III, H., and Crawford, K. (2021). Datasheets for datasets. *Communications of the ACM*, 64(12):86–92.

Gregor, S. and Hevner, A. R. (2013). Positioning and presenting design science research for maximum impact. *MIS Quarterly*, 37(2):337–355.

Gregor, S. and Jones, D. (2007). The anatomy of a design theory. *Journal of the Association for Information Systems*, 8(5):312–335.

Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., and Fritz, M. (2023). Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, CCS '23, pages 79–90. ACM.

Gretel.ai (2024). Synthetic PII finance multilingual (gretelai/synthetic_pii_finance_multilingual). Hugging Face dataset, revision 7b844d16738527a04264f50214cb426a4cea0897. Apache-2.0. `https://huggingface.co/datasets/gretelai/synthetic_pii_finance_multilingual`.

Haber, S. and Stornetta, W. S. (1991). How to time-stamp a digital document. *Journal of Cryptology*, 3(2):99–111.

Herschel, M., Diestelkämper, R., and Ben Lahmar, H. (2017). A survey on provenance: What for? what form? what from? *The VLDB Journal*, 26(6):881–906.

Hevner, A. R., March, S. T., Park, J., and Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1):75–106.

Honnibal, M., Montani, I., Van Landeghem, S., and Boyd, A. (2020). spaCy: Industrial-strength natural language processing in Python. Zenodo. `https://spacy.io`.

ISO/IEC (2023). ISO/IEC 42001:2023 Information Technology – Artificial Intelligence – Management System. International Organization for Standardization. `https://www.iso.org/standard/81230.html`.

Josefsson, S. and Liusvaara, I. (2017). Edwards-Curve digital signature algorithm (EdDSA). RFC 8032, RFC Editor.

Kang, D., Hashimoto, T., Stoica, I., and Sun, Y. (2022). Scaling up trustless DNN inference with zero-knowledge proofs. arXiv preprint arXiv:2210.08674.

Kapoor, S. and Narayanan, A. (2023). Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9):100804.

LangChain Inc. (2023). LangChain and LangGraph. Software, online documentation. `https://www.langchain.com`.

Laurie, B., Langley, A., and Kasper, E. (2013). Certificate Transparency. RFC 6962, RFC Editor.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. arXiv preprint arXiv:2005.11401 (NeurIPS 2020).

Li, M. (2025). From cloud-native to trust-native: A protocol for verifiable multi-agent systems. arXiv preprint arXiv:2507.22077.

Mäntymäki, M., Minkkinen, M., Birkstedt, T., and Viljanen, M. (2022). Defining organizational AI governance. *AI and Ethics*, 2(4):603–609.

Merkle, R. C. (1988). A digital signature based on a conventional encryption function. In *Advances in Cryptology — CRYPTO '87*, pages 369–378. Springer Berlin Heidelberg.

Microsoft (2018). Presidio: Data protection and de-identification SDK. Software, MIT license. `https://github.com/microsoft/presidio`.

Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., Spitzer, E., Raji, I. D., and Gebru, T. (2019). Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency*, FAT* '19, pages 220–229. ACM.

Model Context Protocol (2025). Model Context Protocol specification (2025-06-18). `https://modelcontextprotocol.io/specification/2025-06-18`.

Mökander, J. and Floridi, L. (2021). Ethics-based auditing to develop trustworthy AI. *Minds and Machines*, 31(2):323–327.

Mökander, J., Schuett, J., Kirk, H. R., and Floridi, L. (2023). Auditing large language models: a three-layered approach. *AI and Ethics*, 4(4):1085–1115.

Moreau, L., Clifford, B., Freire, J., Futrelle, J., Gil, Y., Groth, P., Kwasnikowska, N., Miles, S., Missier, P., Myers, J., Plale, B., Simmhan, Y., Stephan, E., and Van den Bussche, J. (2011). The open provenance model core specification (v1.1). *Future Generation Computer Systems*, 27(6):743–756.

Newman, Z., Meyers, J. S., and Torres-Arias, S. (2022). Sigstore: Software signing for everybody. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, CCS '22, pages 2353–2367. ACM.

Novelli, C., Taddeo, M., and Floridi, L. (2023). Accountability in artificial intelligence: what it is and how it works. *AI & SOCIETY*, 39(4):1871–1882.

Ong, I., Almahairi, A., Wu, V., Chiang, W.-L., Wu, T., Gonzalez, J. E., et al. (2024). RouteLLM: Learning to route LLMs with preference data. arXiv preprint arXiv:2406.18665.

OpenSSF (2023). SLSA specification v1.0: Supply-chain levels for software artifacts. `https://slsa.dev/spec/v1.0/`.

OpenTelemetry Authors (2026). OpenTelemetry specification. `https://opentelemetry.io/docs/specs/otel/`.

Papagiannidis, E., Mikalef, P., and Conboy, K. (2025). Responsible artificial intelligence governance: A review and research framework. *The Journal of Strategic Information Systems*, 34(2):101885.

Peffers, K., Tuunanen, T., Rothenberger, M. A., and Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems*, 24(3):45–77.

Portkey AI (2024). Portkey AI gateway. Software, MIT license. `https://github.com/Portkey-AI/gateway`.

Prat, N., Comyn-Wattiau, I., and Akoka, J. (2015). A taxonomy of evaluation methods for information systems artifacts. *Journal of Management Information Systems*, 32(3):229–267.

Protect AI (2023). LLM guard: The security toolkit for LLM interactions. Software, MIT license. `https://github.com/protectai/llm-guard`.

Rai, A. (2020). Explainable AI: from black box to glass box. *Journal of the Academy of Marketing Science*, 48(1):137–141.

Raji, I. D., Smart, A., White, R. N., Mitchell, M., Gebru, T., Hutchinson, B., Smith-Loud, J., Theron, D., and Barnes, P. (2020). Closing the AI accountability gap: defining an end-to-end framework for internal algorithmic auditing. In *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency*, FAT* '20, pages 33–44. ACM.

Rundgren, A., Jordan, B., and Erdtman, S. (2020). JSON canonicalization scheme (JCS). RFC 8785, RFC Editor.

Samuel, J., Mathewson, N., Cappos, J., and Dingledine, R. (2010). Survivable key compromise in software update systems. In *Proceedings of the 17th ACM conference on Computer and communications security*, CCS '10, pages 61–72. ACM.

Schneider, J., Abraham, R., Meske, C., and Vom Brocke, J. (2022). Artificial intelligence governance for businesses. *Information Systems Management*, 40(3):229–249.

Schneier, B. and Kelsey, J. (1999). Secure audit logs to support computer forensics. *ACM Transactions on Information and System Security*, 2(2):159–176.

Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., and Dennison, D. (2015). Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems 28*.

Secure Systems Lab (2021). DSSE: Dead simple signing envelope (specification). GitHub repository. `https://github.com/secure-systems-lab/dsse`.

Seyedkazemi Ardebili, M. (2026). NovaFabric: Tamper-evident, replayable evidence for autonomous AI agent runs. arXiv:2609.12582 [cs.CR]. `https://arxiv.org/abs/2609.12582`.

Shneiderman, B. (2020). Bridging the gap between ethics and practice: Guidelines for reliable, safe, and trustworthy human-centered AI systems. *ACM Transactions on Interactive Intelligent Systems*, 10(4):1–31.

Sonnenberg, C. and vom Brocke, J. (2012). Evaluations in the science of the artificial – reconsidering the build-evaluate pattern in design science research. In *Design Science Research in Information Systems. Advances in Theory and Practice*, pages 381–397. Springer Berlin Heidelberg.

Stelmakh, I., Luan, Y., Dhingra, B., and Chang, M.-W. (2022). ASQA: Factoid questions meet long-form answers. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 8273–8288. Association for Computational Linguistics. Data: din0s/asqa, revision 084060f16b46f3165318f760b2339208b19a0bde, Apache-2.0.

Sun, H., Li, J., and Zhang, H. (2024). zkLLM: Zero knowledge proofs for large language models. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, CCS '24, pages 4405–4419. ACM.

Tabassi, E. (2023). Artificial intelligence risk management framework (AI RMF 1.0). Technical Report NIST AI 100-1, National Institute of Standards and Technology (U.S.).

The Sigstore Authors (2021). cosign: Code signing and transparency for containers and binaries. Software, Apache-2.0 license. `https://github.com/sigstore/cosign`.

Torres-Arias, S., Afzali, H., Kuppusamy, T. K., Curtmola, R., and Cappos, J. (2019). in-toto: Providing farm-to-table guarantees for bits and bytes. In *Proceedings of the 28th USENIX Security Symposium (USENIX Security)*. USENIX Association.

Trace (2026). Eval harness findings: matched PII signed into no-PII receipts, receipt replay and chain gaps, and 8 more issues. GitHub issue #21, fullselfbrowsing/Lattice. Opened by GitHub user Trace824. `https://github.com/fullselfbrowsing/Lattice/issues/21`.

Traceloop (2023). OpenLLMetry: Open-source observability for LLM applications, based on OpenTelemetry. Software, Apache-2.0 license. `https://github.com/traceloop/openllmetry`.

Turlapati, L. (2026). Lattice: Capability runtime SDK for multimodal AI applications (source code). GitHub repository. `https://github.com/fullselfbrowsing/Lattice`.

Venable, J., Pries-Heje, J., and Baskerville, R. (2016). FEDS: a framework for evaluation in design science research. *European Journal of Information Systems*, 25(1):77–89.

vom Brocke, J., Hevner, A., and Maedche, A. (2020). Introduction to design science research. In *Design Science Research. Cases*, pages 1–13. Springer International Publishing.

Walls, J. G., Widmeyer, G. R., and El Sawy, O. A. (1992). Building an information system design theory for vigilant EIS. *Information Systems Research*, 3(1):36–59.

Wieringa, M. (2020). What to account for when accounting for algorithms: a systematic literature review on algorithmic accountability. In *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency*, FAT* '20, pages 1–18. ACM.

Zaratiana, U., Tomeh, N., Holat, P., and Charnois, T. (2024). GLiNER: Generalist model for named entity recognition using bidirectional transformer. In *Proceedings of NAACL 2024 (Volume 1: Long Papers)*, pages 5364–5376. Association for Computational Linguistics.